

holo-graph-compiler

<http://120.209.217.11:31006/toolchain/holo-graph-compiler>

2026.08	下降的大概流程, 变换的IR表示, 下降过程可能的问题
2026.09	初版方案: 1. 快速跑通流程, 无性能要求 2. 模块可插拔替换, IR表达完备, 支持功能拓展 3. 下降采用兜底方案(保证适配性, 不保证性能)

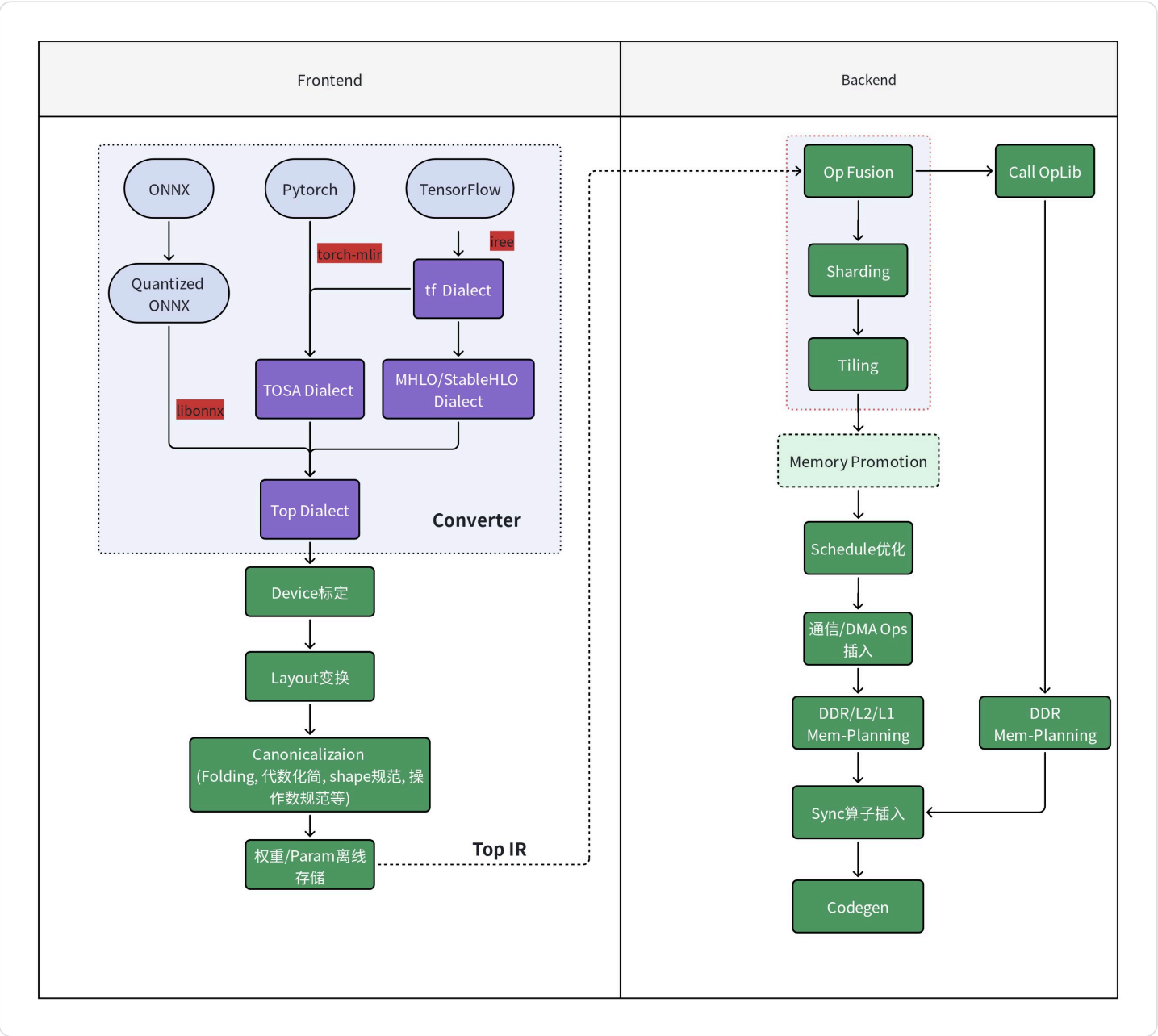
模块	开发周期	联调	测试周期
Converter	0.5人月	编译器内部模块联调	2人月
ShapeInfer/LayoutInfer	1人月	与量化对接联调	1人月
Canonicalization	2人月	与基础编译器对接联调	1人月
权重处理	0.5人月	子网精度达标	3人月
OpFusion	3人月	子网性能调试	4人月
Sharding/Tiling	6人月	VLA整网串联, 精度达标	4人月
Memory Promotion/Schedule优化	3人月	VLA整网性能调试	4人月
DMA算子插入	1人月		
DDR/L2/L1 mem-planning	3人月		
Sync算子插入	3人月		
Codegen	3人月		
其它	4人月		
合计	30人月	合计	18人月

模块待开发功能

1. 简介

图编译器以ONNX, Pytorch, TensorFlow模型为输入, 经过内部一系列变换, 转换成LLVM IR或C/C++ code,
最后通过基础编译器编译, 生成可部署在NPU上的固定格式二进制指令包.

2. 工作流



2.1 框架对接

框架模型经过Converter模块转换成基于Top Dialect表示的mlir文件格式。

2.1.1 ONNX

代码块

```
1 ./build/frontend/tools/onnx-to-mlir ./tests/mnist.onnx
```

代码块

```
1  #loc = loc(unknown)
2  module attributes {top.version = "1.0.0", top.npu_core_num = 4} {
3    func.func @torch_jit(%arg0: tensor<1x1x28x28xf32> loc(unknown)) ->
      tensor<1x10xf32> {
4      %0 = "top.constant"() <{value = dense<[0.0360819325, 0.0303174332,
        -0.014957685, 0.0604293533, 0.0917338133, 0.0978015586, -0.0705472528,
        0.121654645, -0.039008636, -0.0469057448]> : tensor<10xf32>}> : () ->
        tensor<10xf32> loc(#loc1)
5      %1 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<128xf32>}> : () -> tensor<128xf32> loc(#loc2)
6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<10x128xf32>}> : () -> tensor<10x128xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<128x196xf32>}> : () -> tensor<128x196xf32> loc(#loc4)
8      %4 = "top.maxPool"(%arg0) <{kernel_shape = [2, 2], pads = [0, 0, 0, 0],
        strides = [2, 2]}> : (tensor<1x1x28x28xf32>) -> tensor<1x1x14x14xf32>
        loc(#loc5)
9      %5 = "top.reshape"(%4) <{shape = [1, 196]}> : (tensor<1x1x14x14xf32>) ->
        tensor<1x196xf32> loc(#loc6)
10     %6 = "top.gemm"(%5, %3, %1) <{transA = false}> : (tensor<1x196xf32>,
        tensor<128x196xf32>, tensor<128xf32>) -> tensor<1x128xf32> loc(#loc7)
11     %7 = "top.relu"(%6) : (tensor<1x128xf32>) -> tensor<1x128xf32> loc(#loc8)
12     %8 = "top.gemm"(%7, %2, %0) <{transA = false}> : (tensor<1x128xf32>,
        tensor<10x128xf32>, tensor<10xf32>) -> tensor<1x10xf32> loc(#loc9)
13     %9 = "top.softmax"(%8) <{axis = 1 : i64}> : (tensor<1x10xf32>) ->
        tensor<1x10xf32> loc(#loc10)
14     return %9 : tensor<1x10xf32> loc(#loc11)
15   } loc(#loc)
16 } loc(#loc)
17 #loc1 = loc("fc2.bias")
18 #loc2 = loc("fc1.bias")
19 #loc3 = loc("fc2.weight")
```

```
20  #loc4 = loc("fc1.weight")
21  #loc5 = loc("MaxPool_0")
22  #loc6 = loc("Reshape_2")
23  #loc7 = loc("Gemm_3")
24  #loc8 = loc("Relu_4")
25  #loc9 = loc("Gemm_5")
26  #loc10 = loc("Softmax_6")
27  #loc11 = loc("torch_jit_return")
28
```

2.1.2 Pytorch

2.1.3 Tensorflow & Others

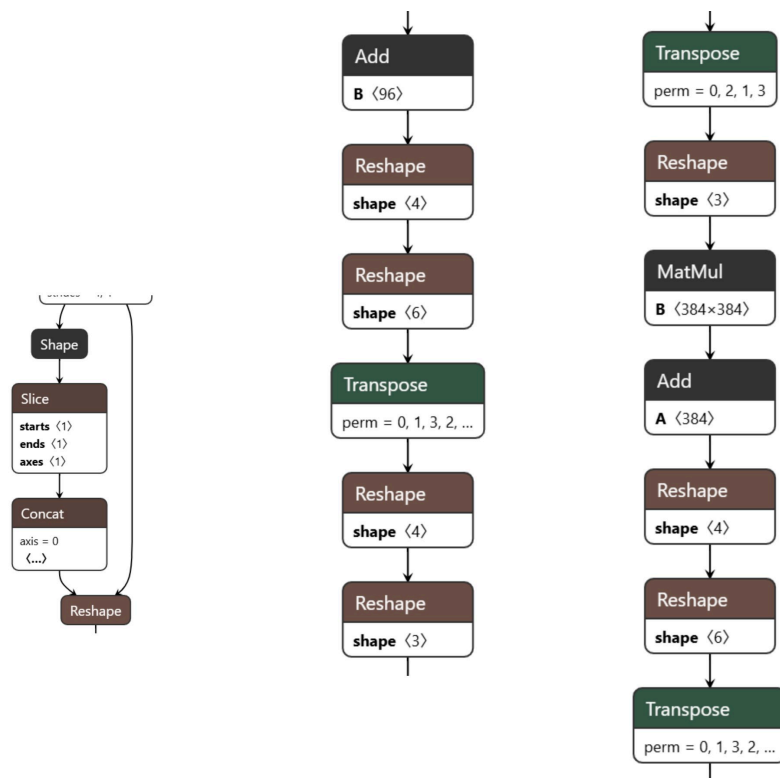
2.2 Canocalization

2.2.1 Folding

1. constant + reshape/transpose/slice
2. constant + 维度broadcast

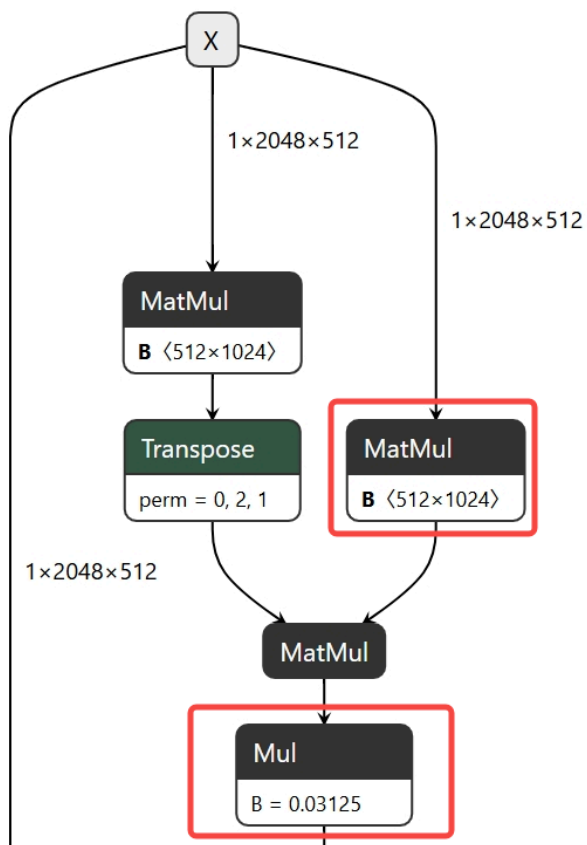
2.2.2 化简

1. slice + concat
2. reshape + reshape
3. transpose + transpose



2.2.3 形状和操作数位置规范

1. matmul/gemm的输入输出形状统一规范成3维等
2. 需要broadcast的操作数统一放置在右侧等



2.2.4 CSE

1. constant去重(影响生命周期, 需要做性能评估)

2.2.5 reshape上浮或下沉

将对形状不敏感的算子跟reshape算子交换位置, 让更多的算子具有相同形状

2.2.6 多输出算子转单输出算子

1. split算子转slice算子

2.2.7 形状, 数据类型和layout的合法检查

2.3 权重离线存储

<https://github.com/safetensors/safetensors>

<https://github.com/hsnyder/safetensors.h>

基于safetensors对权重进行序列化离线存储

2.4 Op Fusion

2.4.1 Fusion定义



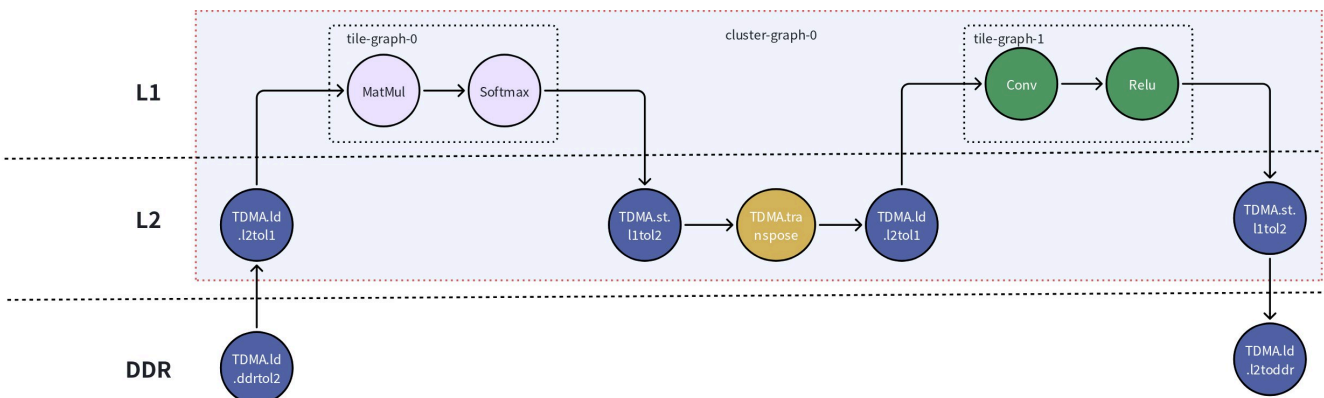
General Op Fusion

在GPU架构中, 通常将多个原本独立执行的算子(如 Conv → BatchNorm → ReLU)合并为一个 CUDA Kernel来执行, 从而消除中间张量在Global Memory中的反复读写, 大幅降低内存带宽开销和Kernel启动开销



Memory-Aware Op Fusion

融合的判定不再仅仅根据算子间的数据交换是否通过global memory, 而是将融合按照 memory level进行划分: 两个操作在哪个memory level进行数据通信, 则称这两个操作在该 memory level融合



按照Memory-Aware Fusion的定义, 可以将NPU架构下的算子融合细分成如下类型



L0 fusion (kernel-graph)

1. 指令间的融合, 指令间均通过register交换数据, 没有对L1/L2/DDR的访问
2. 单个kernel内对register的需求不能超过硬件限制

L1 fusion (tile-graph)

1. 算子间的融合, 所有算子间通过L1交换数据, 没有对L2/DDR的访问
2. tile内的算子采用相同的tiling策略

L2 fusion (cluster-graph)

1. tile间的融合, 所有tile间通过L2交换数据, 没有对DDR的访问
2. cluster内的tile采用相同的sharding策略(减少tile-core间同步, 非严格条件)

将算子进行Memory-Aware Fusion之后, 前端计算图可以表示成如下的嵌套结构

代码块

```
1      1. holo.shard从DDR输入, 输出到DDR, 它由holo.cluster构成
2      2. holo.cluster从L2输入, 输出到L2, 它由holo.tile构成
3      3. holo.tile从L1输入, 输出到L1, 它由具体的算子(指令)构成
4
5      // sharding, DDR
6      %out = holo.shard(%ddr_a, ...) {
7          %l2_a = holo.partial_copy(%ddr_a)
8
9          // tiling, L2
10         %l2_0 = holo.cluster(%l2_a, ...) {
11             %l1_tile_input = holo.partial_copy(%l2_a)
12
13             // tiling, L1
14             %l1_tile_out = holo.tile(%l1_tile_input, ...) {
15                 %l1_x = holo.op.mma(%l1_a, ...)
16                 %l1_y = holo.op.relu(%l1_x)
17                 holo.return %l1_y
18             }
19             %l2_tile_out = holo.partial_insert(%l1_tile_out)
20
21             holo.return %l2_tile_out
```

```

22     }
23
24     %ddr_out = holo.partial_insert(%l2_0)
25     holo.return %ddr_out
26 }

```

2.4.2 Fusion类型比较

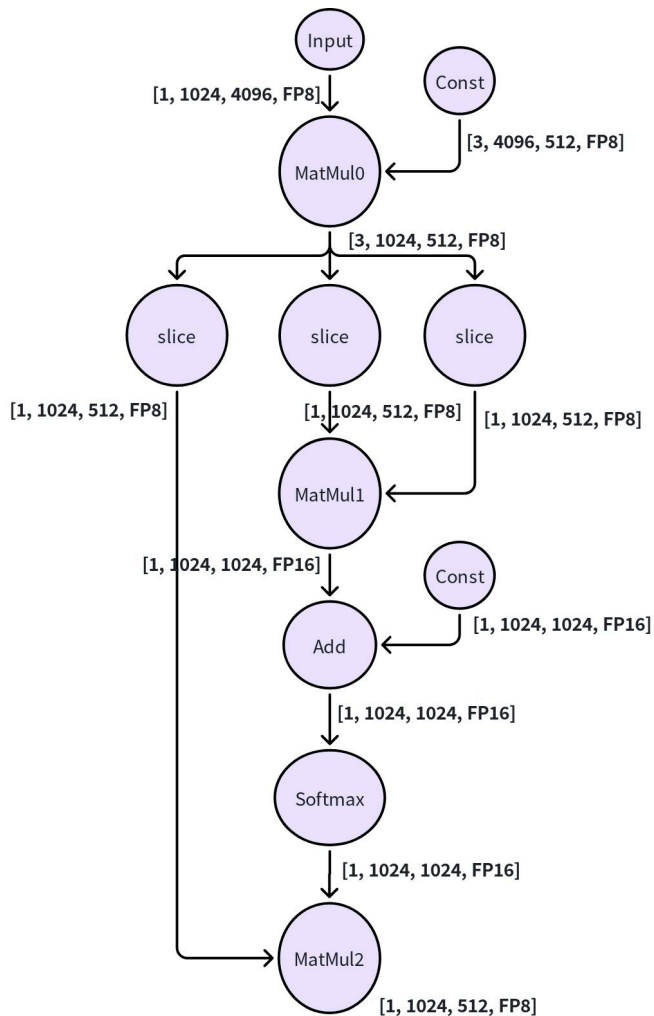
General Op Fusion	1. 融合策略简单, 只需要判断算子间的中间结果是否经过DDR 2. 不需要决策fusion op group内算子如何通过L1/L2交换数据 3. 适合对接算子库, 不与sharding/tiling耦合
Memory-Aware Op Fusion	1. 融合策略复杂, 需要决策fusion op group内任意相邻算子间在哪一层memory level(DDR/L2/L1)融合 2. 通常需要结合sharding/tiling一起建模, 处理的算子规模更大, 融合效果更好 3. 层次清晰, 方便mem-planning和同步分析

-  SPMD下的诉求
1. 需要充分利用大L2 buffer的优势, 数据尽可能的驻留在L2, 所以从内存层次出发, 关注的边界点是DDR(找到一个尽可能大的op region, 数据从DDR进, 输出到DDR, 中间结果不会spill到DDR)
 2. Op Fusion完成后能对接算子库
 3. Op Fusion能够与sharding/tiling解耦, 方便模块迭代和多方案设计

基于上述诉求, 倾向采用General Op Fusion.

 General Op Fusion在没有考虑sharding/tiling的情况下, 是否会导致融合策略受限或偏保守

以下是一个常见数据规模的attention结构, 假设将它作为一个融合的op group进行分析.



代码块

```

1  /*
2   对于上述模型结构，将其融合成一个op group
3   tensor core: 单指令 16*(4/8)*16 OPs / cycle
4   vector core: 单指令 32 elements / cycle
5   DDR带宽: 1TB/s * 0.7 = 716.8 GB/s
6
7   以下计算和数据加载使用一个tile core完成
8  */
9
10 // 从DDR加载的数据总量
11 1*1024*4096*1 + 3*4096*512*1 + 1*1024*1024*2 = 12MB
12 cycles: 12MB / (716.8GB/s) * 2G (cycles/s) = 33482 cycles
13
14 // 写出到DDR的数据总量
15 1*1024*512*1 = 0.5MB
16 cycles: 0.5MB / (716.8GB/s) * 2G (cycles/s) = 1395 cycles
17
18 // tensor core
19 MatMul0: 3*1024*512*4096 OPs

```

```

20  MatMul1: 1024*1024*512 OPs
21  MatMul2: 1024*512*1024*2 OPs
22  cycles: 8053063680 OPs / (16*8*16*64) = 61440 cycles
23
24  // vector core
25  Add: 1024*1024 = 1048576 elements
26  Softmax: (1024*1024) * 10 = 10485760 elements
27  cycles: 11534336 elements / (32*64) = 5632 cycles
28
29
30  // L2的峰值占用
31  /*
32   做如下假设:
33   1. MatMul0的结果完全驻留在L2
34   2. Matmul1+Add+Softmax在L1融合, 每次切片是[m = 32, n = 1024]
35   3. Softmax与Matmul2在L2融合, 它的结果全部驻留在L2
36   4. 所有的权重全部驻留在L2
37  */
38  所有中间结果的L2占用:
39  3*1024*512 + 1024*1024*2 = 3.5MB
40  L2峰值占用:
41  3.5MB + 12MB = 15.5MB

```

从上面分析来看, 即使不做细粒度的切分, L2的峰值占用相比于它的物理上限, 仍然是相对宽裕的. 所以, 在General Op Fusion模式下, 仍然可以实施比较激进的mega-kernel融合, 同时保证sharding/tiling阶段搜索空间有可行解.

2.4.3 Fusion Pattern定义

2.4.3.1 IR

方案1 IR文件可读性差; 方案2结构清晰, netron可视化看不到group节点内部

代码块

```

1  // attribute标记fusion group
2  func.func @matmul_relu(...) {
3      ...
4      %0 = top.matmul(%a, %b) {fusion_group = 0 : i64}
5      %1 = top.relu(%0) {fusion_group = 0 : i64}
6      ...
7  }
8
9  // 自定义group op
10 // XLA mhlo.fusion, IREE flow.dispatch.region(一次kernel launch)

```

```

11 //
    https://github.com/openxla/xla/blob/2a4c54254d11333b0054734e852be26f44899123/xla/hlo/translate/mhlo_to_hlo/tests/fusion.mlir#L32
12 // https://github.com/iree-
    org/iree/blob/9983d03e4f1a097414addd99cae964fb542d4ba5/compiler/src/iree/compiler/DispatchCreation/test/form_dispatch_regions.mlir#L3175
13 func.func @matmul_relu(...) {
14     ...
15     %c = top.group(%a, %b) {
16         %0 = top.matmul(%a, %b)
17         %1 = top.relu(%0)
18         top.return %1
19     }
20     ...
21 }

```

代码块

```

1  #loc = loc(unknown)
2  module attributes {top.core_num = 4 : i64, top.version = "1.0.0"} {
3      func.func @torch_jit(%arg0: tensor<1x1x28x28xf32> loc(unknown)) ->
        tensor<1x10xf32> {
4          %0 = "top.constant"() <{value = dense<[0.0360819325, 0.0303174332,
            -0.014957685, 0.0604293533, 0.0917338133, 0.0978015586, -0.0705472528,
            0.121654645, -0.039008636, -0.0469057448]> : tensor<10xf32>}> : () ->
            tensor<10xf32> loc(#loc1)
5          %1 = "top.constant"() <{value = dense_resource<__elided__> :
            tensor<128xf32>}> : () -> tensor<128xf32> loc(#loc2)
6          %2 = "top.constant"() <{value = dense_resource<__elided__> :
            tensor<10x128xf32>}> : () -> tensor<10x128xf32> loc(#loc3)
7          %3 = "top.constant"() <{value = dense_resource<__elided__> :
            tensor<128x196xf32>}> : () -> tensor<128x196xf32> loc(#loc4)
8          %4 = top.fusion {
9              %8 = "top.maxPool"(%arg0) <{kernel_shape = [2, 2], pads = [0, 0, 0, 0],
                strides = [2, 2]}> : (tensor<1x1x28x28xf32>) -> tensor<1x1x14x14xf32>
                loc(#loc5)
10             top.return %8 : tensor<1x1x14x14xf32> loc(#loc5)
11         } : tensor<1x1x14x14xf32> loc(#loc5)
12         %5 = top.fusion {
13             %8 = "top.reshape"(%4) <{shape = [1, 196]}> : (tensor<1x1x14x14xf32>) ->
                tensor<1x196xf32> loc(#loc6)
14             top.return %8 : tensor<1x196xf32> loc(#loc6)
15         } : tensor<1x196xf32> loc(#loc6)
16         %6 = top.fusion {

```

```

17      %8 = "top.gemm"(%5, %3, %1) <{alpha = 1 : i64, beta = 1 : i64, transA =
      false, transB = false}> : (tensor<1x196xf32>, tensor<128x196xf32>,
      tensor<128xf32>) -> tensor<1x128xf32> loc(#loc7)
18      %9 = "top.relu"(%8) : (tensor<1x128xf32>) -> tensor<1x128xf32> loc(#loc8)
19      top.return %9 : tensor<1x128xf32> loc(#loc7)
20  } : tensor<1x128xf32> loc(#loc7)
21  %7 = top.fusion {
22      %8 = "top.gemm"(%6, %2, %0) <{alpha = 1 : i64, beta = 1 : i64, transA =
      false, transB = false}> : (tensor<1x128xf32>, tensor<10x128xf32>,
      tensor<10xf32>) -> tensor<1x10xf32> loc(#loc9)
23      %9 = "top.softmax"(%8) <{axis = 1 : i64}> : (tensor<1x10xf32>) ->
      tensor<1x10xf32> loc(#loc10)
24      top.return %9 : tensor<1x10xf32> loc(#loc9)
25  } : tensor<1x10xf32> loc(#loc9)
26  return %7 : tensor<1x10xf32> loc(#loc11)
27  } loc(#loc)
28  } loc(#loc)
29  #loc1 = loc("fc2.bias")
30  #loc2 = loc("fc1.bias")
31  #loc3 = loc("fc2.weight")
32  #loc4 = loc("fc1.weight")
33  #loc5 = loc("MaxPool_0")
34  #loc6 = loc("Reshape_2")
35  #loc7 = loc("Gemm_3")
36  #loc8 = loc("Relu_4")
37  #loc9 = loc("Gemm_5")
38  #loc10 = loc("Softmax_6")
39  #loc11 = loc("torch_jit_return")

```

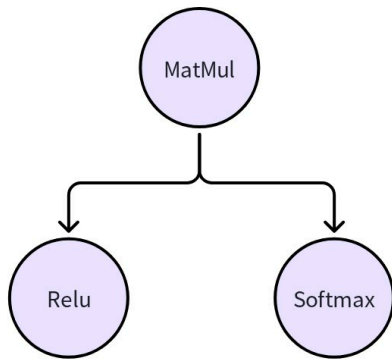
2.4.3.2 多输出支持



融合方案A

1. 融合pattern只能有一个输出
2. sharding&tiling size由输出tensor决定, 并自底向上推导

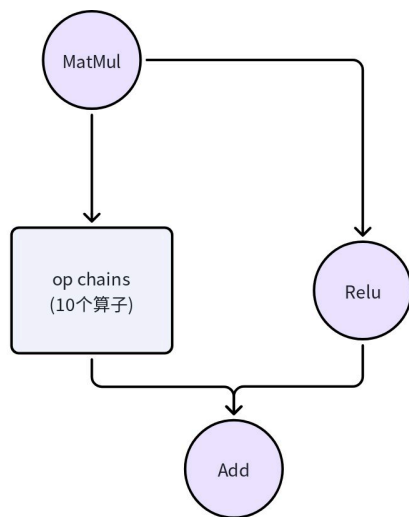
方案A要求单输出的原因: sharding&tiling时, root算子唯一确定



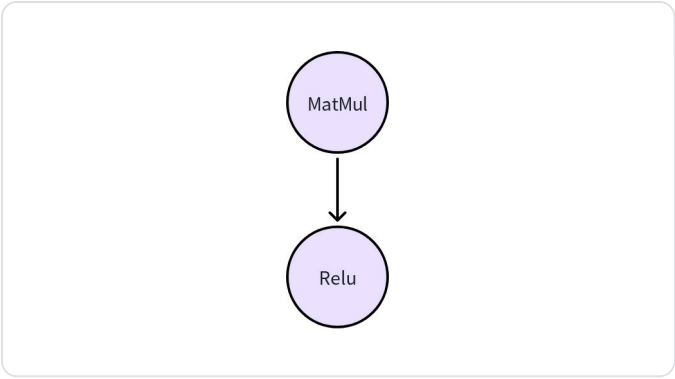
融合方案B

1. 融合pattern支持多输出
2. 每个op支持sharding/tiling的forward/backward, sharding&tiling size可由任意中间op决定(一般选择核心算子)

1. 多user, 且user间的汇聚节点深度比较大, 方案A很难覆盖



2. 由于MatMul的输出只有[M, N]两个轴, 自底向上推导无法表达K的切分

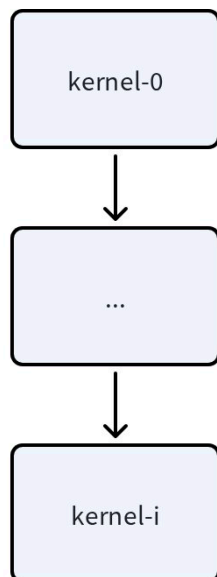


2.4.3.3 自定义算子pattern描述

模式	细节
PDLL	代码块 <pre>1 #include "mhlo/IR/hlo_ops.td" 2 #include "MyDialect/MyOps.td" // 定义 my.dot_bias 3 4 Constraint HasOneUse(v: Value) [{ 5 return success(v.hasOneUse()); 6 }]; 7 8 // 类型一致 9 Constraint SameType(a: Type, b: Type) [{ 10 return success(a == b); 11 }]; 12 13 Pattern MhloDotAddToFused { 14 let lhs: Value; 15 let rhs: Value; 16 let bias: Value; 17 18 let dot = op<mhlo.dot>(lhs, rhs) -> (ty: Type); 19 apply HasOneUse(dot.0); 20 21 let add = op<mhlo.add>(dot.0, bias) -> (ty2: Type); 22 apply SameType(ty, ty2); 23 24 rewrite add with { 25 let fused = op<my.dot_bias>(lhs, rhs, bias) -> (ty); 26 replace add with fused; 27 }; 28 }</pre>

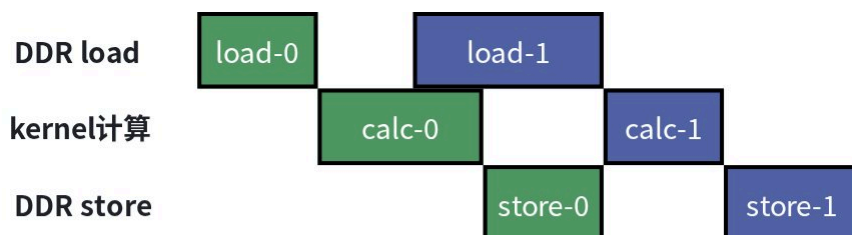
2.4.4 Fusion Kernel的运行模式

按照SPMD的约定, 一个kernel运行是独占64个tile cores, 那么kernel会被串行调度执行



在这种调度模式下, 除了DDR, 其它所有资源都是kernel独占的, 内存分配, 同步分析以及调度优化都基本限定在kernel内部, 但无法实现DDR数据的预取, 只能通过kernel内部的流水编排, 去隐藏DDR的加载延迟.

如果是memory-bound kernel, DDR数据预取收益明显的情况下, 是否应该打破调度边界, 允许下一个kernel的DDR加载穿插到上一个kernel的计算逻辑内. 比如上一个kernel计算强度很大, L2的占用率较低, 下一个kernel是memory-bound.

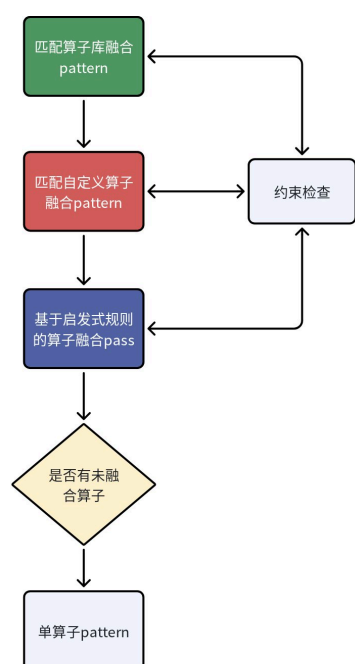


2.4.5 Fusion策略

在3.2节中, 我们调研了主流的开源AI编译器算子融合方案, 可以归结为如下几种方案:

1. 算子库支持的算子融合pattern
2. 自定义算子融合pattern
3. 基于启发式规则制定算子融合策略

我们采用的融合策略如下:



2.4.6 遗留问题

? 图编译器L0融合存在的问题

1. L0 fusion需要考虑寄存器分配(基础编译器负责), 同时L0 fusion情况会影响L1的分配
2. 过早执行L0-fusion, 没有寄存器分配信息; 过晚执行L0-fusion, 会影响L1-fusion的正确性

代码块

```
1 // L0融合
2 scf.for n...
3     scf.for m...
```

```

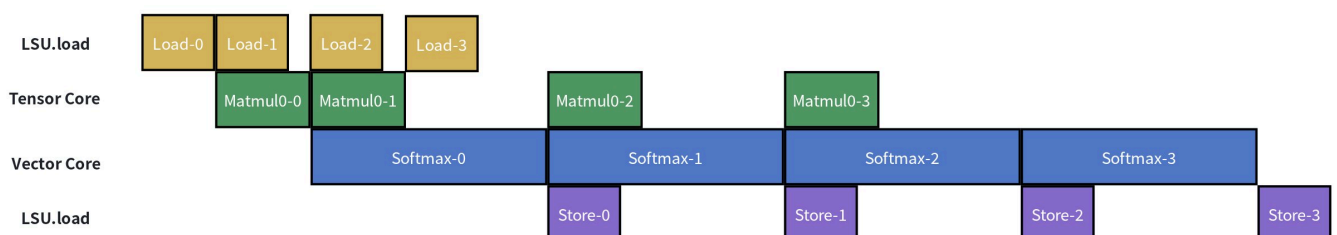
4      %acc_a = init()
5      scf.for k...
6          %tile_a = tile.lsu.load.32x32(%slice_A)
7          %tile_b = tile.lsu.load.32x32(%slice_B)
8          %acc = tile.mma.32x32x32(%tile_a, %tile_b, %acc_a)
9
10     %acc_b = tile.vector.relu(%acc_a)
11     tile.lsu.store.32x32(%acc_b)
12
13     // L0不融合
14     %l1_out = memref.alloc()
15     scf.for n...
16         scf.for m...
17             %acc_a = init()
18             scf.for k...
19                 %tile_a = tile.lsu.load.32x32(%A)
20                 %tile_b = tile.lsu.load.32x32(%B)
21                 %acc = tile.mma.32x32x32(%tile_a, %tile_b, %acc_a)
22                 tile.lsu.store.32x32(%acc_a, %l1_out)
23
24     scf.for n...
25         scf.for m...
26             %acc_relu = tile.lsu.load.32x32(%l1_out)
27             %acc_out = tile.vector.relu(%acc_relu)
28             tile.lsu.store.32x32(%acc_out, %l1_out)

```

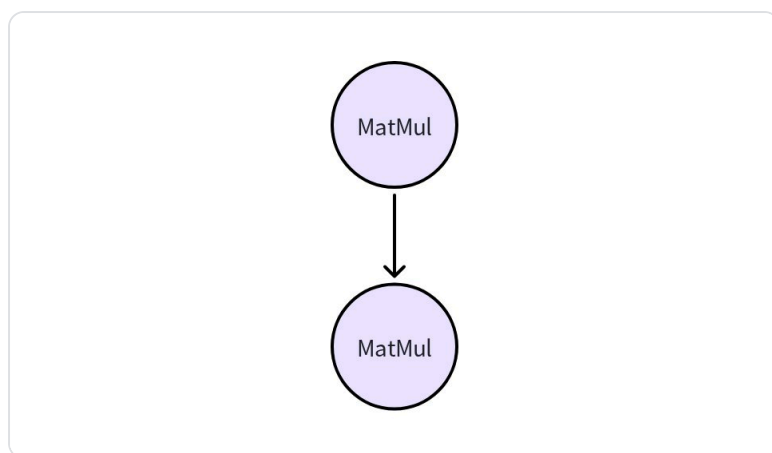
? 是否可以认为L1 fusion内的所有算子同时是L0 fusion的

- 在下面场景中, Tensor Core/LSU存在大量空闲, 但Vector Core被占满, 如果softmax的下游是Tensor Core或者LSU算子可以继续L0融合, 如果是Vector Core, 则无需融合

L0 Fusion



2. Tensor Core + Tensor Core算子搭配



2.5 Sharding&Tiling

在fusion的基础上, 决策融合操作间每次通信的数据位置(tiling-axes)和数据量(tiling-size)

2.5.1 Sharding

沿tensor的部分轴进行切分, 切分后的数据分发到不同的设备/计算核心上

2.5.1.1 Multi-Chip表示

shard dialect(mesh dialect) <https://mlir.llvm.org/docs/Dialects/Shard/>, 支持设备网格和集合通信

代码块

```
1  shard.grid @device_mesh(shape = 4x16x16)
2
3  // tensor切分规则: 第0维沿mesh轴0, 第1维沿mesh轴1
4  %sharding = shard.sharding @device_mesh split_axes = [[0], [1]] :
    !shard.sharding
5
6  // 在tensor上应用切分规则
7  %0 = shard.shard %arg0 to %sharding : tensor<4x8xf32>
8
9  // case 0
10 shard.grid @device_mesh(shape = 2x2)
11 func.func @apply(%arg0: tensor<4x8xf32>) -> tensor<4x8xf32> {
```

```

12  %sharding = shard.sharding @device_mesh split_axes = [[0], [1]] :
    !shard.sharding
13  %0 = shard.shard %arg0 to %sharding : tensor<4x8xf32>
14  return %0 : tensor<4x8xf32>
15  }
16
17  // case 1, 非均匀切分
18  shard.grid @device_mesh(shape = 4)
19  %s = shard.sharding @device_mesh
20  split_axes = [[], [0]]
21  // 张量第1维被切成4段, 区间为[0, 2), [2, 5), [5, 9), [9, 14)
22  // size分别为 2, 3, 4, 5
23  sharded_dims_offsets = [0, 2, 5, 9, 14]
24  : !shard.sharding
25  %0 = shard.shard %arg0 to %s : tensor<4x14xf32>

```

? 如何partition, 切换到single-chip上的表示, 同时维护全局tensor信息

2.5.1.2 Single-Chip表示

前端表达

代码块

```

1  %mm = top.matmul {schedule_config = <sharding = [4, 2, 1]>}
2  (%A: tensor<128x256xf32>, %B: tensor<256x64xf32>) -> tensor<128x64xf32>

```

后端表达

代码块

```

1  #map_b = affine_map<(d0) -> (d0 * 3)>
2  #map_m = affine_map<(d0) -> (d0 * 128)>
3  #map_n = affine_map<(d0) -> (d0 * 256)>
4  #map_b_min = affine_map<(d0)[s0] -> (3, s0 - d0)>
5
6  func.func @batch_matmul(%A: memref<5x256x128xf32, 0>,
7                          %B: memref<1x128x512xf32, 0>,
8                          %C: memref<5x256x512xf32, 0>) {

```

```

9      %c0 = arith.constant 0 : index
10     %c1 = arith.constant 1 : index
11     %c5 = arith.constant 5 : index
12     %c32 = arith.constant 32 : index
13     %c128 = arith.constant 128 : index
14     %c256 = arith.constant 256 : index
15
16     scf.forall (%b_core, %m_core, %n_core) in (2, 2, 2) {
17         %b0 = affine.apply #map_b(%b_core)
18         %m0 = affine.apply #map_m(%m_core)
19         %n0 = affine.apply #map_n(%n_core)
20         %b_size = affine.min #map_b_min(%b0) [%c5]
21
22         %A_L2 = memref.alloc(%b_size) : memref<?x128x128xf32, 1>
23         %B_L2 = memref.alloc() : memref<1x128x256xf32, 1>
24         %C_L2 = memref.alloc(%b_size) : memref<?x128x256xf32, 1>
25
26         %A_ddr = memref.subview %A[%b0, %m0, 0] [%b_size, 128, 128] [1, 1, 1]
27             : memref<5x256x128xf32, 0>
28             to memref<?x128x128xf32, strided<[32768, 128, 1], offset: ?>, 0>
29         %B_ddr = memref.subview %B[%c0, 0, %n0] [1, 128, 256] [1, 1, 1]
30             : memref<1x128x512xf32, 0>
31             to memref<1x128x256xf32, strided<[65536, 512, 1], offset: ?>, 0>
32         %C_ddr = memref.subview %C[%b0, %m0, %n0] [%b_size, 128, 256] [1, 1, 1]
33             : memref<5x256x512xf32, 0>
34             to memref<?x128x256xf32, strided<[131072, 512, 1], offset: ?>, 0>
35
36         memref.copy %A_ddr, %A_L2
37             : memref<?x128x128xf32, strided<[32768, 128, 1], offset: ?>, 0>
38             to memref<?x128x128xf32, 1>
39         memref.copy %B_ddr, %B_L2
40             : memref<1x128x256xf32, strided<[65536, 512, 1], offset: ?>, 0>
41             to memref<1x128x256xf32, 1>
42         memref.copy %C_ddr, %C_L2
43             : memref<?x128x256xf32, strided<[131072, 512, 1], offset: ?>, 0>
44             to memref<?x128x256xf32, 1>
45
46         %A_L1 = memref.alloc() : memref<32x32xf32, 2>
47         %B_L1 = memref.alloc() : memref<32x32xf32, 2>
48         %C_L1 = memref.alloc() : memref<32x32xf32, 2>
49
50         scf.for %bb = %c0 to %b_size step %c1 {
51             scf.for %ii = %c0 to %c128 step %c32 {
52                 scf.for %jj = %c0 to %c256 step %c32 {
53                     %C_l2_tile = memref.subview %C_L2[%bb, %ii, %jj] [1, 32, 32] [1, 1,
54
1]
                    : memref<?x128x256xf32, 1>

```

```

55         to memref<32x32xf32, strided<[256, 1], offset: ?>, 1>
56         memref.copy %C_l2_tile, %C_L1
57         : memref<32x32xf32, strided<[256, 1], offset: ?>, 1>
58         to memref<32x32xf32, 2>
59
60     scf.for %kk = %c0 to %c128 step %c32 {
61         %A_l2_tile = memref.subview %A_L2[%bb, %ii, %kk] [1, 32, 32] [1,
1, 1]
62         : memref<?x128x128xf32, 1>
63         to memref<32x32xf32, strided<[128, 1], offset: ?>, 1>
64         %B_l2_tile = memref.subview %B_L2[%c0, %kk, %jj] [1, 32, 32] [1,
1, 1]
65         : memref<1x128x256xf32, 1>
66         to memref<32x32xf32, strided<[256, 1], offset: ?>, 1>
67         memref.copy %A_l2_tile, %A_L1
68         : memref<32x32xf32, strided<[128, 1], offset: ?>, 1>
69         to memref<32x32xf32, 2>
70         memref.copy %B_l2_tile, %B_L1
71         : memref<32x32xf32, strided<[256, 1], offset: ?>, 1>
72         to memref<32x32xf32, 2>
73
74         linalg.matmul ins(%A_L1, %B_L1 : memref<32x32xf32, 2>,
memref<32x32xf32, 2>)
75         outs(%C_L1 : memref<32x32xf32, 2>)
76     }
77
78     memref.copy %C_L1, %C_l2_tile
79     : memref<32x32xf32, 2>
80     to memref<32x32xf32, strided<[256, 1], offset: ?>, 1>
81 }
82 }
83 }
84
85 memref.copy %C_L2, %C_ddr
86 : memref<?x128x256xf32, 1>
87 to memref<?x128x256xf32, strided<[131072, 512, 1], offset: ?>, 0>
88
89 memref.dealloc %A_L1 : memref<32x32xf32, 2>
90 memref.dealloc %B_L1 : memref<32x32xf32, 2>
91 memref.dealloc %C_L1 : memref<32x32xf32, 2>
92 memref.dealloc %A_L2 : memref<?x128x128xf32, 1>
93 memref.dealloc %B_L2 : memref<1x128x256xf32, 1>
94 memref.dealloc %C_L2 : memref<?x128x256xf32, 1>
95 } {mapping = [#holo.tile_core_mapping<x>,
96 #holo.tile_core_mapping<y>,
97 #holo.tile_core_mapping<z>]}
98 return

```

2.5.1.3 interface定义

代码块

```

1  struct ShardingOption {
2      int64_t core_num;
3      // 比如core_num = 8, 0轴切4份, 1轴切2份, 2轴不切
4      // [4, 2, 1]
5      std::vector<int64_t> shard_num;
6  };
7
8  struct ShardingIterationInfo {
9      int64_t iter_index;
10 };
11
12 /*
13  接口支持算子多输出
14 */
15
16 bool isValidInputShardingOption(const ShardingOption& option, int64_t
operand_index);
17 bool isValidOutputShardingOption(const std::vector<ShardingOption>& options);
18 // 获取root算子的输出tensor的sharding option
19 std::vector<ShardingOption> getOpSharding(ShardingIterationInfo& info);
20 // 支持backward, 从输出tensor推导输入tensor的sharding options
21 std::vector<ShardingOption> getInputTensorSharding(const
std::vector<ShardingOption>& output_sharding_options);
22 // 支持forward, 从输入tensor推导输出tensor的sharding options
23 std::vector<ShardingOption> getOutputTensorSharding(const
std::vector<ShardingOption>& input_sharding_options);
24

```

2.5.2 Tiling

对分配到单个core上数据块进行切分，切分后的数据slice沿时间维度依次执行

2.5.2.1 Tiling表示

前端表达

代码块

```
1  %mm = top.matmul {schedule_config = <sharding = [4, 2, 1], tiling = [32, 32, 0]>}  
2  (%A: tensor<128x256xf32>, %B: tensor<256x64xf32>) -> tensor<128x64xf32>
```

代码块

```
1  #map_b = affine_map<(d0) -> (d0 * 2)>  
2  #map_m = affine_map<(d0) -> (d0 * 128)>  
3  #map_n = affine_map<(d0) -> (d0 * 256)>  
4  
5  // memory space  
6  // DDR: 0, L2: 1, L1: 2  
7  func.func @batch_matmul(%A: memref<4x256x128xf32, 0>,  
8  %B: memref<1x128x512xf32, 0>,  
9  %C: memref<4x256x512xf32, 0>) {  
10   %c0 = arith.constant 0 : index  
11   %c1 = arith.constant 1 : index  
12   %c2 = arith.constant 2 : index  
13   %c32 = arith.constant 32 : index  
14   %c128 = arith.constant 128 : index  
15   %c256 = arith.constant 256 : index  
16  
17   scf.forall (%b_core, %m_core, %n_core) in (2, 2, 2) {  
18     %b0 = affine.apply #map_b(%b_core)  
19     %m0 = affine.apply #map_m(%m_core)  
20     %n0 = affine.apply #map_n(%n_core)  
21  
22     %A_dds = memref.subview %A[%b0, %m0, 0] [2, 128, 128] [1, 1, 1] :  
memref<4x256x128xf32, 0>  
23     to memref<2x128x128xf32, strided<[32768, 128, 1], offset: ?>, 0>  
24     %B_dds = memref.subview %B[0, 0, %n0] [1, 128, 256] [1, 1, 1] :  
memref<1x128x512xf32, 0>  
25     to memref<1x128x256xf32, strided<[65536, 512, 1], offset: ?>, 0>  
26     %C_dds = memref.subview %C[%b0, %m0, %n0] [2, 128, 256] [1, 1, 1] :  
memref<4x256x512xf32, 0>  
27     to memref<2x128x256xf32, strided<[131072, 512, 1], offset: ?>, 0>  
28  
29     // TDMA: DDR -> L2, [2, 128, 128] x [1, 128, 256]  
30     %A_L2 = memref.alloc() : memref<2x128x128xf32, 1>  
31     %B_L2 = memref.alloc() : memref<1x128x256xf32, 1>  
32     %C_L2 = memref.alloc() : memref<2x128x256xf32, 1>  
33     memref.copy %A_dds, %A_L2
```

```

34         : memref<2x128x128xf32, strided<[32768, 128, 1], offset: ?>, 0> to
memref<2x128x128xf32, 1>
35     memref.copy %B_dds, %B_L2
36         : memref<1x128x256xf32, strided<[65536, 512, 1], offset: ?>, 0> to
memref<1x128x256xf32, 1>
37     memref.copy %C_dds, %C_L2
38         : memref<2x128x256xf32, strided<[131072, 512, 1], offset: ?>, 0> to
memref<2x128x256xf32, 1>
39
40     // L1
41     %A_L1 = memref.alloc() : memref<32x32xf32, 2>
42     %B_L1 = memref.alloc() : memref<32x32xf32, 2>
43     %C_L1 = memref.alloc() : memref<32x32xf32, 2>
44
45     scf.for %bb = %c0 to %c2 step %c1 {
46         scf.for %ii = %c0 to %c128 step %c32 {
47             scf.for %jj = %c0 to %c256 step %c32 {
48
49                 // TDMA: L2 -> L1, C
50                 %C_l2_tile = memref.subview %C_L2[%bb, %ii, %jj] [1, 32, 32] [1,
1, 1]
51                     : memref<2x128x256xf32, 1> to memref<32x32xf32, strided<[256,
1], offset: ?>, 1>
52                 memref.copy %C_l2_tile, %C_L1
53                     : memref<32x32xf32, strided<[256, 1], offset: ?>, 1> to
memref<32x32xf32, 2>
54
55                 // K轴乘累加
56                 scf.for %kk = %c0 to %c128 step %c32 {
57                     // TDMA: L2 -> L1, A, B
58                     %A_l2_tile = memref.subview %A_L2[%bb, %ii, %kk] [1, 32, 32]
[1, 1, 1] : memref<2x128x128xf32, 1>
59                         to memref<32x32xf32, strided<[128, 1], offset: ?>, 1>
60                     %B_l2_tile = memref.subview %B_L2[0, %kk, %jj] [1, 32, 32] [1,
1, 1] : memref<1x128x256xf32, 1>
61                         to memref<32x32xf32, strided<[256, 1], offset: ?>, 1>
62
63                     memref.copy %A_l2_tile, %A_L1
64                         : memref<32x32xf32, strided<[128, 1], offset: ?>, 1> to
memref<32x32xf32, 2>
65                     memref.copy %B_l2_tile, %B_L1
66                         : memref<32x32xf32, strided<[256, 1], offset: ?>, 1> to
memref<32x32xf32, 2>
67
68                     linalg.matmul ins(%A_L1, %B_L1 : memref<32x32xf32, 2>,
memref<32x32xf32, 2>)
69                         outs(%C_L1 : memref<32x32xf32, 2>)

```

```

70         }
71
72         // 结果回写到L2
73         memref.copy %C_L1, %C_l2_tile : memref<32x32xf32, 2>
74             to memref<32x32xf32, strided<[256, 1], offset: ?>, 1>
75     }
76 }
77 }
78
79 // 结果回写到DDR
80 memref.copy %C_L2, %C_ddr : memref<2x128x256xf32, 1>
81     to memref<2x128x256xf32, strided<[131072, 512, 1], offset: ?>, 0>
82
83 memref.dealloc %A_L1 : memref<32x32xf32, 2>
84 memref.dealloc %B_L1 : memref<32x32xf32, 2>
85 memref.dealloc %C_L1 : memref<32x32xf32, 2>
86 memref.dealloc %A_L2 : memref<2x128x128xf32, 1>
87 memref.dealloc %B_L2 : memref<1x128x256xf32, 1>
88 memref.dealloc %C_L2 : memref<2x128x256xf32, 1>
89 } {mapping = [#holo.tile_core_mapping<x>,
90             #holo.tile_core_mapping<y>,
91             #holo.tile_core_mapping<z>]}
92 return
93 }

```

2.5.2.2 interface定义

代码块

```

1  struct TilingOption {
2      // [32, 32, 0]
3      std::vector<int64_t> tiling_size;
4  };
5
6  struct TilingIterationInfo {
7      int64_t iter_index;
8  };
9
10 /*
11  接口支持算子多输出
12  */
13
14 bool isValidInputTilingOption(const TilingOption& option, int64_t
    operand_index);

```

```

15  bool isValidOutputTilingOption(const std::vector<TilingOption>& options);
16  // 获取root算子的输出tensor的tiling option
17  std::vector<TilingOption> getOpTiling(TilingIterationInfo& info);
18  // 支持backward, 从输出tensor推导输入tensor的tiling options
19  std::vector<TilingOption> getInputTensorTiling(const
std::vector<TilingOption>& output_tiling_options);
20  // 支持forward, 从输入tensor推导输出tensor的tiling option
21  std::vector<TilingOption> getOutputTensorTiling(const
std::vector<TilingOption>& output_tiling_options);
22

```

2.5.3 Sharding&Tiling策略

在3.3节, 我们调研了主流的开源AI编译器, 它们采用的Sharding和Tiling策略可以归结为2类:

1. 启发式: 选取root op, root op的切分策略基于启发式规则, 经递推传播, 实现融合子图内所有op的切分
2. AutoSchedule: 构建搜索空间, 使用智能算法或机器学习算法选取候选解, 上机评测, 更新数据库/cost-model; 推理时查询数据库或走cost-model评估获取融合子图的最佳切分方案

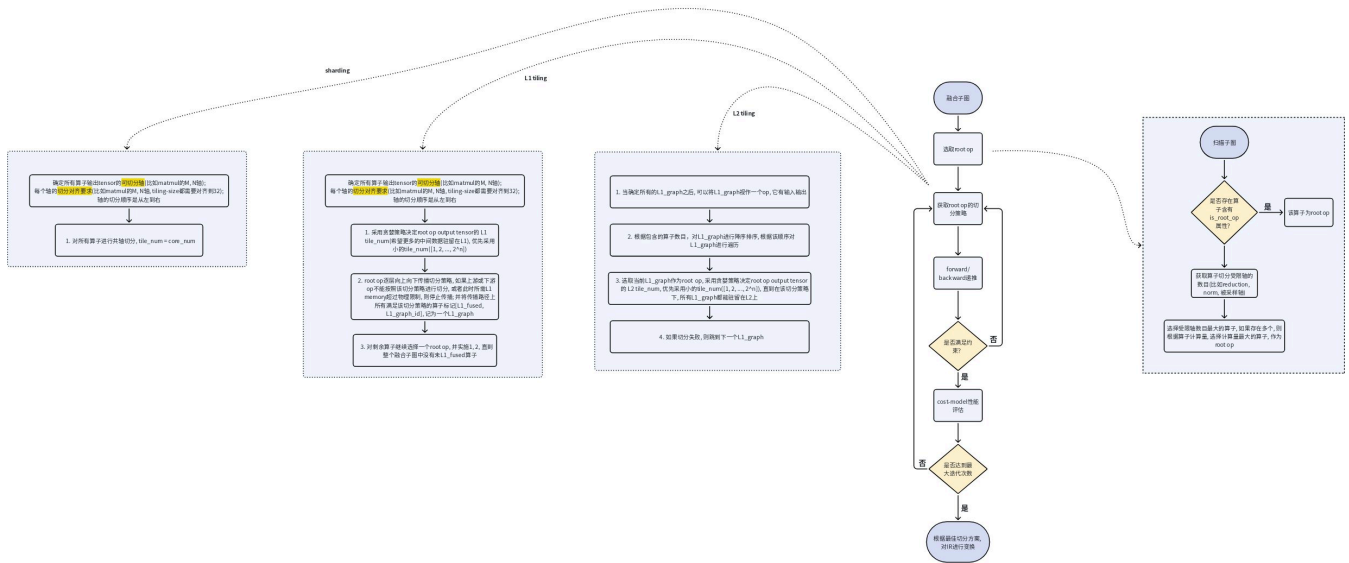
Sharding&Tiling策略需要处理的共性问题:

1. 将计算任务切分到指定数目的NPU cores, 保证负载均衡
2. 利用中间buffer, 尽可能减少对high-latency memory的访问
3. 将计算和IO进行overlap, 隐藏IO的开销

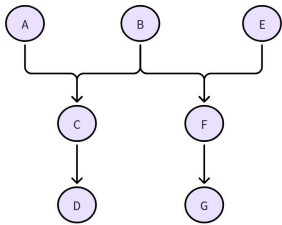
策略设计时考量如下:

1. 对于一个计算任务, 可以指定所用的core num, 并完成均匀切分
2. 数据在每一层memory level的tiling size是可调的, 保证切分空间的完备性
3. ping-pong流水可以通过控制mem-planning和同步实现

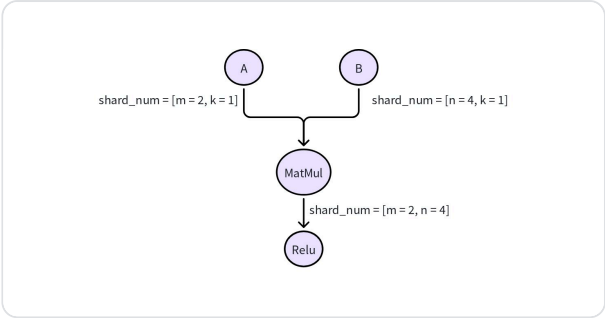
策略描述如下 [\[图编译器串讲\] fusion & sharding & tiling](#)



L2和DDR的区别	除了size和latency的差异, 从tile core的视角看, L2跟DDR没有区别。 所以可以将L2作为DDR的补充, 在kernel内部, L2的角色如下: 1. 作为DDR和L1的缓冲 2. 当算子A, B无法共轴切分到多个tile, 那么A的计算完成后, 结果在L2汇集, 再进行B的计算
	1. 假设Fusion已经完成, 一个Fusion op内包含多个融合算子 2. 依据规则, 将算子进行优先级排序, 优先级最高的算子, 作为root算子 3. 从root算子出发, 获取它的最佳切分方式(切分数目大于等于core_num), 向上向下传播该切分方式, 将传播路径上所有接受该切分方式的算子移动到一个Tile op的region内 4. 对剩余算子继续进行2, 3操作, 直到没有剩余算子
	1. Tile op内的所有算子workload首先会切分到不同core上, 且所有的中间数据均在L1, Register上 2. 如果core tile切分后, L1能够放下所有的数据, 则输入, 权重直接从DDR加载到L1, 不经过L2 3. 如果L1不能放下所有数据, 则计算最多能够切分多少份, L1最多能放多少份 4. 再以最大切分数目放入L2

已决策	待决策
除了算子库算子融合pattern和自定义融合pattern, 其它算子的融合策略跟tiling, sharding一并决策	是否应该将fusion, sharding, tiling作为一个整体进行决策 1. 每一层单独决策, 后续模块缺乏兜底方案 2. 由于相互耦合, 每一层单独决策, 很难构建合适的策略
	哪些数据需要放在L2 1. DDR -> L2 与 L2 -> L1 & 计算的流水数据 2. 被多个core共享的数据, 或被重复使用的数据 3. 算子在L2融合
需要建模两个算子是否可以在Register融合, 方便准确评估L1的使用量	
两个算子是否可以放进同一个group, 主要依据是它们是否可以共轴切分, 且partition_num >= core_num	1. 计算当前group内满足L1 size要求的所有partition_num 2. 如下, 假如已完成C的切分, 向上递推到B, 此时F的一路切分已确定, 另一路未确定; 需要增加一个sibling推导, 根据算子特点和已切分partition_num, 推测另一路的partition_num. <pre>graph TD; A((A)) --> C((C)); B((B)) --> C; B --> F((F)); E((E)) --> F; C --> D((D)); F --> G((G));</pre> 3. reuse or recompute SPMD执行在8个tile core上, 将matmul的计算切分到8个core上, m切两份, n切4份, 那么对应的matmul的左矩阵只需要切2份, 右矩阵切4份, 分别repeat 4次和2次, 分发到8个core上. 假设左矩阵是A算子的输出, 右矩阵是B算子的输出, 那么有如下几种选择: 1. A只在2个core上执行, 计算完后, 再1-to-4广播, B同理; 1. 0-3 core重复计算第一份数据, 4-7 core重复计算第二份数据, 无数据广播; B同理

2. 将A的计算拆分成8份, 分别在8个core上计算, 再allGather; B同理



- 1. $total_partition_num = L2_tiling_num * core_num * L1_tiling_num$, 在已知 $total_partition_num$ 和 $core_num$ 的情况下, 如何实现 $L2_tiling_num$ 和 $L1_tiling_num$ 的最优分配
S0: 隐藏DDR->L2的延迟是最重要的, 所以基于 cost-model, 令每个L2切片是DDR->L2和L2->L1 & 计算的latency一致, 即找到最佳的 $L2_tiling_num$.
S1: $L1_tiling_num$ 能够保证Tile Op所需L1 size 不大于物理限制
- 2. 目前L2-fusion和L1-fusion均基于自定义 pattern, 自动融合pattern待实现
- 3. 如何确定给每个Tile Op分配多少个Tile Core
- 4. 是否限定同一个Fusion Op内的Tile Op是串行执行的; 如果是, 则无法实现数据流

- 1. 两级Fusion: 将Fusion Op内的算子进行二次fusion, 组成Tile Op, **Tile Op是单输出op**, Tile Op与Tile Op之间通过L2交换数据, L2-fusion(DDR进, DDR出, 支持多输出), L1-fusion(L2进, L2出, 支持单输出)
- 2. 三级切分: L2-tiling(DDR tensor在L2的切片), core-sharding(L2 tensor多核并行), L1-tiling(单core数据在L1的切片)
- 3. 同一个Fusion Op内的Tile Op可以指定不同的Tile Core Cluster, 因为不同的Tile Op之间是通过L2交换数据, 它们可以处于不同的Tile Core Cluster. 这样的好处是:
 - a. 有些Tile Op形状比较小, 可以使用较少数目的Tile Core
 - b. 可以实现dataflow

2.5.4 示例

前端mlir: 携带切分配置

```
代码块
1
2 module attributes {top.core_num = 64 : i64, top.version = "1.0.0"} {
```

```

3    func.func @AttentionGraphFixed(%arg0: tensor<1x2048x512xf32> loc(unknown)) -
> tensor<1x2048x1024xf32> {
4      %0 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<1x2048x2048xf32>}> : () -> tensor<1x2048x2048xf32> loc(#loc1)
5      %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
> tensor<1x1x1xf32> loc(#loc2)
6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc4)
8      %4 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc5)
9      %5 = top.fusion {
10      %6 = "top.fusionInput"(%arg0) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32> loc(#loc6)
11      %7 = "top.fusionInput"(%0) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32> loc(#loc6)
12      %8 = "top.fusionInput"(%1) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
loc(#loc6)
13      %9 = "top.fusionInput"(%2) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
14      %10 = "top.fusionInput"(%3) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
15      %11 = "top.fusionInput"(%4) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
16      %12 = top.tile {
17      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc7)
18      %19 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1,
1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>
loc(#loc7)
19      %20 = "top.matmul"(%18, %19) <{transB = false}> {is_root = 1 : i64} :
(tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}>,
tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1,
1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>) ->

```

```

tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 16]}}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 32, 2]}},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>
loc(#loc7)

```

```

20      %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16]}}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 2, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2]}}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 32]}}>) -> tensor<1x2048x1024xf32> loc(#loc7)

```

```

21      top.return %21 : tensor<1x2048x1024xf32> loc(#loc7)

```

```

22      } : tensor<1x2048x1024xf32> loc(#loc7)

```

```

23      %13 = top.tile {

```

```

24      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1,
16, 1]}}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 16 : i64, shard_nums = [1, 4, 1]}},
partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc8)

```

```

25      %19 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
[1, 1, 1]}}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 2]},
core_sharding = {replicate_num = 4 : i64, shard_nums = [1, 1, 16]}},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>
loc(#loc8)

```

```

26      %20 = "top.matmul"(%18, %19) <{transB = false}> :
(tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 16, 1]}}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 16 : i64, shard_nums = [1, 4, 1]}},
partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}>,
tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
[1, 1, 1]}}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 2]},
core_sharding = {replicate_num = 4 : i64, shard_nums = [1, 1, 16]}},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>) ->
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 16, 1]}}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 2]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 4, 16]}},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>
loc(#loc8)

```

```

27      %21 = "top.transpose"(%20) <{perm = [0, 2, 1]}> {is_root = 1 : i64} :
(tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 16, 1]}}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 2]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 4, 16]}},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>) ->
tensor<1x1024x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 16]}}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 16, 4]}},

```

```

partition_config = {replicate_num = 1 : i64, shard_nums = [1, 32, 64]}}>
loc(#loc9)
28      %22 = "top.tileOutput"(%21) : (tensor<1x1024x2048xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 2, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 16, 4]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 32, 64]}}>) -> tensor<1x1024x2048xf32> loc(#loc8)
29      top.return %22 : tensor<1x1024x2048xf32> loc(#loc8)
30      } : tensor<1x1024x2048xf32> loc(#loc8)
31      %14 = top.tile {
32      %18 = "top.tileInput"(%7) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc6)
33      %19 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32,
{L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, core_sharding =
{replicate_num = 64 : i64, shard_nums = [1, 1, 1]}, partition_config =
{replicate_num = 64 : i64, shard_nums = [1, 1, 1]}}> loc(#loc6)
34      %20 = "top.tileInput"(%12) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc6)
35      %21 = "top.tileInput"(%13) : (tensor<1x1024x2048xf32>) ->
tensor<1x1024x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 64 : i64, shard_nums = [1, 1, 1]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 1]}}>
loc(#loc6)
36      %22 = "top.matmul"(%20, %21) <{transB = false}> :
(tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>,
tensor<1x1024x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 64 : i64, shard_nums = [1, 1, 1]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 1]}}>) ->
tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc6)

```

```

37      %23 = "top.mul"(%22, %19) : (tensor<1x2048x2048xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}}>, tensor<1x1x1xf32, {L1_tiling = {replicate_num = 1
: i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64,
shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 64 : i64, shard_nums
= [1, 1, 1]}, partition_config = {replicate_num = 64 : i64, shard_nums = [1,
1, 1]}}>) -> tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64,
shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64,
1]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc10)
38      %24 = "top.add"(%23, %18) : (tensor<1x2048x2048xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}}>, tensor<1x2048x2048xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}}>) -> tensor<1x2048x2048xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}}> loc(#loc11)
39      %25 = "top.softmax"(%24) <{axis = 2 : i64}> {is_root = 1 : i64} :
(tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>) ->
tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc12)
40      %26 = "top.tileOutput"(%25) : (tensor<1x2048x2048xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}}>) -> tensor<1x2048x2048xf32> loc(#loc6)
41      top.return %26 : tensor<1x2048x2048xf32> loc(#loc6)
42      } : tensor<1x2048x2048xf32> loc(#loc6)
43      %15 = top.tile {
44      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},

```

```

core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]]}>
loc(#loc13)
45      %19 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1,
1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]]}>
loc(#loc13)
46      %20 = "top.matmul"(%18, %19) <{transB = false}> {is_root = 1 : i64} :
(tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]]}>,
tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1,
1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]]}>) ->
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 16]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 32, 2]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 32]]}>
loc(#loc13)
47      %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 2, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 32]]}>) -> tensor<1x2048x1024xf32> loc(#loc13)
48      top.return %21 : tensor<1x2048x1024xf32> loc(#loc13)
49      } : tensor<1x2048x1024xf32> loc(#loc13)
50      %16 = top.tile {
51      %18 = "top.tileInput"(%14) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]]}>
loc(#loc14)
52      %19 = "top.tileInput"(%15) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]]}>
loc(#loc14)
53      %20 = "top.matmul"(%18, %19) <{transB = false}> {is_root = 1 : i64} :
(tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},

```

```

partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}},
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}> ->
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 16]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 32, 2]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>
loc(#loc14)
54      %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 2, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 32]}}>) -> tensor<1x2048x1024xf32> loc(#loc14)
55      top.return %21 : tensor<1x2048x1024xf32> loc(#loc14)
56      } : tensor<1x2048x1024xf32> loc(#loc14)
57      %17 = "top.fusionOutput"(%16) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc6)
58      top.return %17 : tensor<1x2048x1024xf32> loc(#loc6)
59      } : tensor<1x2048x1024xf32> loc(#loc6)
60      return %5 : tensor<1x2048x1024xf32> loc(#loc15)
61  } loc(#loc)
62  } loc(#loc)
63

```

代码块

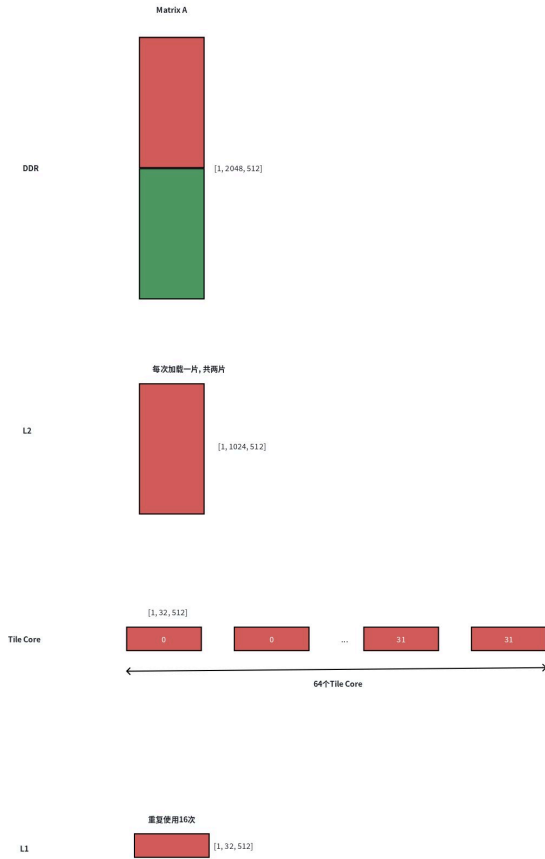
```

1
2  // MatMul
3  // matrix A: tensor<1x2048x512xf32>
4  // matrix B: tensor<1x512x1024xf32>
5  // matrix C: tensor<1x2048x1024xf32>
6
7  {L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
8  core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
9  L1_tiling = {replicate_num = 16 : i64, shard_nums = [1, 1, 1]}
10
11  {L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
12  core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
13  L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 16]}
14
15  // matrix A:
16  shape = [1, 2048, 512]

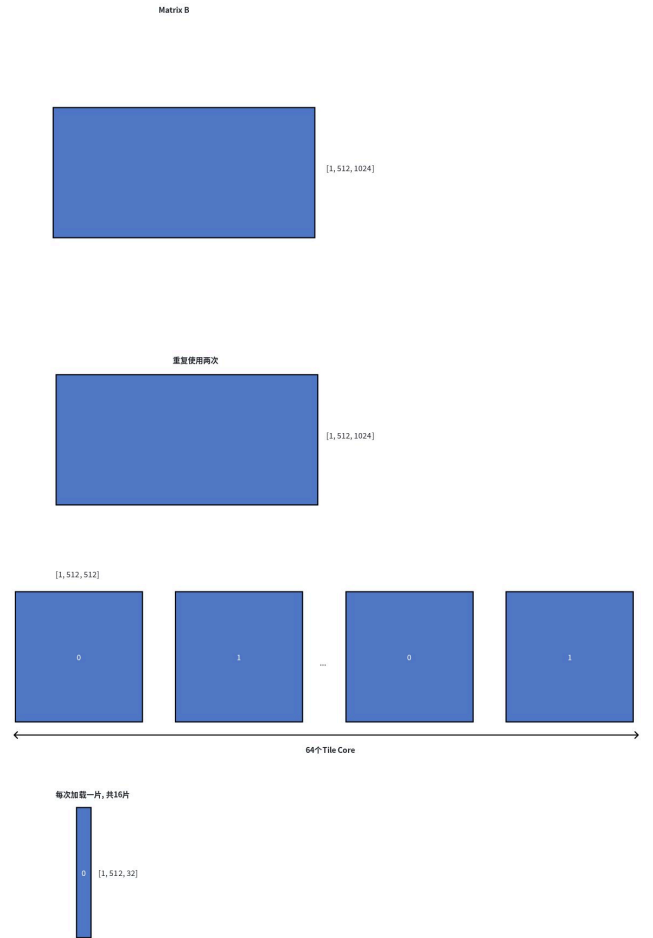
```

```
17 L2_tiling: rep = 1, shards = [1, 2, 1], sub-shape = [1, 1024, 512]
18 core_sharding: rep = 2, shards = [1, 32, 1], sub-shape = [1, 32, 512]
19 L1_tiling: rep = 16, shards = [1, 1, 1], sub-shape = [1, 32, 512]
20
21 // matrix B:
22 shape = [1, 512, 1024]
23 L2_tiling: rep = 2, shards = [1, 1, 1], sub-shape = [1, 512, 1024]
24 core_sharding: rep = 32, shards = [1, 1, 2], sub-shape = [1, 512, 512]
25 L1_tiling: rep = 1, shards = [1, 1, 16], sub-shape = [1, 512, 32]
26
27 {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 16]},
28  L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
29  core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 32, 2]}}
30
31 // matrix C:
32 shape = [1, 2048, 512]
33 L2_tiling: rep = 1, shards = [1, 2, 1], sub-shape = [1, 1024, 512]
34 core_sharding: rep = 1, shards = [1, 32, 1], sub-shape = [1, 32, 512]
35 L1_tiling: rep = 1, shards = [1, 1, 16], sub-shape = [1, 32, 32]
36
37
38
39
```

```
{L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
L1_tiling = {replicate_num = 16 : i64, shard_nums = [1, 1, 1]}}
```



```
{L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 16]}}
```



后端mlir: 下降成scf表示

代码块

```
1  #loc = loc(unknown)
2  module attributes {top.core_num = 64 : i64, top.version = "1.0.0"} {
3    func.func @AttentionGraphFixed(%arg0: tensor<1x2048x512xf32> loc(unknown)) -
      > tensor<1x2048x1024xf32> {
4      %0 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<1x2048x2048xf32>}> : () -> tensor<1x2048x2048xf32> loc(#loc1)
5      %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
        > tensor<1x1x1xf32> loc(#loc2)
6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc4)
8      %4 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc5)
```

```

9      %5 = top.fusion {
10      %6 = "top.fusionInput"(%arg0) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32> loc(#loc6)
11      %7 = "top.fusionInput"(%0) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32> loc(#loc6)
12      %8 = "top.fusionInput"(%1) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
loc(#loc6)
13      %9 = "top.fusionInput"(%2) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
14      %10 = "top.fusionInput"(%3) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
15      %11 = "top.fusionInput"(%4) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
16      %12 = top.tile {
17      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc7)
18      %19 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc7)
19      %20 = tensor.empty() : tensor<1x2048x1024xf32, {}> loc(#loc7)
20      %c0 = arith.constant 0 : index loc(#loc7)
21      %c0_0 = arith.constant 0 : index loc(#loc7)
22      %c0_1 = arith.constant 0 : index loc(#loc7)
23      %c1 = arith.constant 1 : index loc(#loc7)
24      %c2048 = arith.constant 2048 : index loc(#loc7)
25      %c1024 = arith.constant 1024 : index loc(#loc7)
26      %c1_2 = arith.constant 1 : index loc(#loc7)
27      %c1024_3 = arith.constant 1024 : index loc(#loc7)
28      %c1024_4 = arith.constant 1024 : index loc(#loc7)
29      %21 = scf.for %arg1 = %c0 to %c1 step %c1_2 iter_args(%arg2 = %20) ->
(tensor<1x2048x1024xf32, {}>) {
30      %23 = scf.for %arg3 = %c0_0 to %c2048 step %c1024_3 iter_args(%arg4
= %arg2) -> (tensor<1x2048x1024xf32, {}>) {
31      %24 = scf.for %arg5 = %c0_1 to %c1024 step %c1024_4
iter_args(%arg6 = %arg4) -> (tensor<1x2048x1024xf32, {}>) {
32      %25 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc7)
33      %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
1024, 512] [1, 1, 1] : tensor<1x2048x512xf32, {}> to tensor<1x1024x512xf32,
{}> loc(#loc7)
34      %26 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc7)
35      %extracted_slice_5 = tensor.extract_slice %19[%arg1, 0, %arg5]
[1, 512, 1024] [1, 1, 1] : tensor<1x512x1024xf32, {}> to
tensor<1x512x1024xf32, {}> loc(#loc7)
36      %extracted_slice_6 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to
tensor<1x1024x1024xf32, {}> loc(#loc7)

```

```

37         %27 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 1024,
1024) step (1, 32, 512) shared_outs(%arg10 = %extracted_slice_6) ->
(tensor<1x1024x1024xf32, {}>) {
38         %extracted_slice_7 = tensor.extract_slice
%extracted_slice[%arg7, %arg8, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x1024x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc7)
39         %extracted_slice_8 = tensor.extract_slice
%extracted_slice_5[%arg7, 0, %arg9] [1, 512, 512] [1, 1, 1] :
tensor<1x512x1024xf32, {}> to tensor<1x512x512xf32, {}> loc(#loc7)
40         %extracted_slice_9 = tensor.extract_slice %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x1024x1024xf32, {}> to
tensor<1x32x512xf32, {}> loc(#loc7)
41         %c0_10 = arith.constant 0 : index loc(#loc7)
42         %c0_11 = arith.constant 0 : index loc(#loc7)
43         %c0_12 = arith.constant 0 : index loc(#loc7)
44         %c1_13 = arith.constant 1 : index loc(#loc7)
45         %c32 = arith.constant 32 : index loc(#loc7)
46         %c512 = arith.constant 512 : index loc(#loc7)
47         %c1_14 = arith.constant 1 : index loc(#loc7)
48         %c32_15 = arith.constant 32 : index loc(#loc7)
49         %c32_16 = arith.constant 32 : index loc(#loc7)
50         %28 = scf.for %arg11 = %c0_10 to %c1_13 step %c1_14
iter_args(%arg12 = %extracted_slice_9) -> (tensor<1x32x512xf32, {}>) {
51         %29 = scf.for %arg13 = %c0_11 to %c32 step %c32_15
iter_args(%arg14 = %arg12) -> (tensor<1x32x512xf32, {}>) {
52         %30 = scf.for %arg15 = %c0_12 to %c512 step %c32_16
iter_args(%arg16 = %arg14) -> (tensor<1x32x512xf32, {}>) {
53         %extracted_slice_17 = tensor.extract_slice
%extracted_slice_7[%arg11, %arg13, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x32x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc7)
54         %extracted_slice_18 = tensor.extract_slice
%extracted_slice_8[%arg11, 0, %arg15] [1, 512, 32] [1, 1, 1] :
tensor<1x512x512xf32, {}> to tensor<1x512x32xf32, {}> loc(#loc7)
55         %extracted_slice_19 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x512xf32,
{}> to tensor<1x32x32xf32, {}> loc(#loc7)
56         %31 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_17, %extracted_slice_18 : tensor<1x32x512xf32, {}>,
tensor<1x512x32xf32, {}>) outs(%extracted_slice_19 : tensor<1x32x32xf32, {}>) -
> tensor<1x32x32xf32, {}> loc(#loc7)
57         %inserted_slice_20 = tensor.insert_slice %31 into
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x32xf32, {}>
into tensor<1x32x512xf32, {}> loc(#loc7)
58         scf.yield %inserted_slice_20 : tensor<1x32x512xf32, {}>
loc(#loc7)
59         } {tiling_level = "L1_tiling"} loc(#loc7)
60         scf.yield %30 : tensor<1x32x512xf32, {}> loc(#loc7)

```

```

61         } {tiling_level = "L1_tiling"} loc(#loc7)
62         scf.yield %29 : tensor<1x32x512xf32, {}> loc(#loc7)
63         } {tiling_level = "L1_tiling"} loc(#loc7)
64         scf.forall.in_parallel {
65             tensor.parallel_insert_slice %28 into %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x32x512xf32, {}> into
tensor<1x1024x1024xf32, {}> loc(#loc7)
66         } loc(#loc7)
67         } {tiling_level = "core_sharding"} loc(#loc7)
68         %inserted_slice = tensor.insert_slice %27 into %arg6[%arg1,
%arg3, %arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x1024x1024xf32, {}> into
tensor<1x2048x1024xf32, {}> loc(#loc7)
69         scf.yield %inserted_slice : tensor<1x2048x1024xf32, {}>
loc(#loc7)
70         } {tiling_level = "L2_tiling"} loc(#loc7)
71         scf.yield %24 : tensor<1x2048x1024xf32, {}> loc(#loc7)
72         } {tiling_level = "L2_tiling"} loc(#loc7)
73         scf.yield %23 : tensor<1x2048x1024xf32, {}> loc(#loc7)
74         } {tiling_level = "L2_tiling"} loc(#loc7)
75         %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {}>) ->
tensor<1x2048x1024xf32> loc(#loc7)
76         top.return %22 : tensor<1x2048x1024xf32> loc(#loc7)
77         } : tensor<1x2048x1024xf32> loc(#loc7)
78         %13 = top.tile {
79             %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc8)
80             %19 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc8)
81             %20 = tensor.empty() : tensor<1x2048x1024xf32> loc(#loc8)
82             %21 = tensor.empty() : tensor<1x1024x2048xf32, {}> loc(#loc9)
83             %c0 = arith.constant 0 : index loc(#loc9)
84             %c0_0 = arith.constant 0 : index loc(#loc9)
85             %c0_1 = arith.constant 0 : index loc(#loc9)
86             %c1 = arith.constant 1 : index loc(#loc9)
87             %c1024 = arith.constant 1024 : index loc(#loc9)
88             %c2048 = arith.constant 2048 : index loc(#loc9)
89             %c1_2 = arith.constant 1 : index loc(#loc9)
90             %c512 = arith.constant 512 : index loc(#loc9)
91             %c2048_3 = arith.constant 2048 : index loc(#loc9)
92             %22 = scf.for %arg1 = %c0 to %c1 step %c1_2 iter_args(%arg2 = %21) ->
(tensor<1x1024x2048xf32, {}>) {
93                 %24 = scf.for %arg3 = %c0_0 to %c1024 step %c512 iter_args(%arg4 =
%arg2) -> (tensor<1x1024x2048xf32, {}>) {
94                     %25 = scf.for %arg5 = %c0_1 to %c2048 step %c2048_3
iter_args(%arg6 = %arg4) -> (tensor<1x1024x2048xf32, {}>) {
95                         %26 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc8)

```

```

96         %extracted_slice = tensor.extract_slice %18[%arg1, %arg5, 0] [1,
2048, 512] [1, 1, 1] : tensor<1x2048x512xf32, {}> to tensor<1x2048x512xf32,
{}> loc(#loc8)
97         %27 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc8)
98         %extracted_slice_4 = tensor.extract_slice %19[%arg1, 0, %arg3]
[1, 512, 512] [1, 1, 1] : tensor<1x512x1024xf32, {}> to tensor<1x512x512xf32,
{}> loc(#loc8)
99         %extracted_slice_5 = tensor.extract_slice %20[%arg1, %arg5,
%arg3] [1, 2048, 512] [1, 1, 1] : tensor<1x2048x1024xf32> to
tensor<1x2048x512xf32> loc(#loc8)
100        %extracted_slice_6 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, 512, 2048] [1, 1, 1] : tensor<1x1024x2048xf32, {}> to
tensor<1x512x2048xf32, {}> loc(#loc9)
101        %28 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 512,
2048) step (1, 32, 512) shared_outs(%arg10 = %extracted_slice_6) ->
(tensor<1x512x2048xf32, {}>) {
102            %extracted_slice_7 = tensor.extract_slice
%extracted_slice[%arg7, %arg9, 0] [1, 512, 512] [1, 1, 1] :
tensor<1x2048x512xf32, {}> to tensor<1x512x512xf32, {}> loc(#loc8)
103            %extracted_slice_8 = tensor.extract_slice
%extracted_slice_4[%arg7, 0, %arg8] [1, 512, 32] [1, 1, 1] :
tensor<1x512x512xf32, {}> to tensor<1x512x32xf32, {}> loc(#loc8)
104            %extracted_slice_9 = tensor.extract_slice
%extracted_slice_5[%arg7, %arg9, %arg8] [1, 512, 32] [1, 1, 1] :
tensor<1x2048x512xf32> to tensor<1x512x32xf32> loc(#loc8)
105            %extracted_slice_10 = tensor.extract_slice %arg10[%arg7,
%arg8, %arg9] [1, 32, 512] [1, 1, 1] : tensor<1x512x2048xf32, {}> to
tensor<1x32x512xf32, {}> loc(#loc9)
106            %c0_11 = arith.constant 0 : index loc(#loc10)
107            %c1_12 = arith.constant 1 : index loc(#loc10)
108            %c0_13 = arith.constant 0 : index loc(#loc9)
109            %c0_14 = arith.constant 0 : index loc(#loc9)
110            %c0_15 = arith.constant 0 : index loc(#loc9)
111            %c1_16 = arith.constant 1 : index loc(#loc9)
112            %c32 = arith.constant 32 : index loc(#loc9)
113            %c512_17 = arith.constant 512 : index loc(#loc9)
114            %c1_18 = arith.constant 1 : index loc(#loc9)
115            %c32_19 = arith.constant 32 : index loc(#loc9)
116            %c32_20 = arith.constant 32 : index loc(#loc9)
117            %29 = scf.for %arg11 = %c0_13 to %c1_16 step %c1_18
iter_args(%arg12 = %extracted_slice_10) -> (tensor<1x32x512xf32, {}>) {
118                %30 = scf.for %arg13 = %c0_14 to %c32 step %c32_19
iter_args(%arg14 = %arg12) -> (tensor<1x32x512xf32, {}>) {
119                    %31 = scf.for %arg15 = %c0_15 to %c512_17 step %c32_20
iter_args(%arg16 = %arg14) -> (tensor<1x32x512xf32, {}>) {

```

```

120         %extracted_slice_21 = tensor.extract_slice
%extracted_slice_7[%arg11, %arg15, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x512x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc8)
121         %extracted_slice_22 = tensor.extract_slice
%extracted_slice_8[%arg11, 0, %arg13] [1, 512, 32] [1, 1, 1] :
tensor<1x512x32xf32, {}> to tensor<1x512x32xf32, {}> loc(#loc8)
122         %extracted_slice_23 = tensor.extract_slice
%extracted_slice_9[%arg11, %arg15, %arg13] [1, 32, 32] [1, 1, 1] :
tensor<1x512x32xf32> to tensor<1x32x32xf32> loc(#loc8)
123         %32 = linalg.batch_matmul ins(%extracted_slice_21,
%extracted_slice_22 : tensor<1x32x512xf32, {}>, tensor<1x512x32xf32, {}>)
outs(%extracted_slice_23 : tensor<1x32x32xf32>) -> tensor<1x32x32xf32>
loc(#loc8)
124         %extracted_slice_24 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x512xf32,
{}> to tensor<1x32x32xf32, {}> loc(#loc9)
125         %transposed = linalg.transpose ins(%32 :
tensor<1x32x32xf32>) outs(%extracted_slice_24 : tensor<1x32x32xf32, {}>)
permutation = [0, 2, 1] {is_root = true} loc(#loc9)
126         %inserted_slice_25 = tensor.insert_slice %transposed
into %arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] :
tensor<1x32x32xf32, {}> into tensor<1x32x512xf32, {}> loc(#loc9)
127         scf.yield %inserted_slice_25 : tensor<1x32x512xf32, {}>
loc(#loc9)
128         } {tiling_level = "L1_tiling"} loc(#loc9)
129         scf.yield %31 : tensor<1x32x512xf32, {}> loc(#loc9)
130         } {tiling_level = "L1_tiling"} loc(#loc9)
131         scf.yield %30 : tensor<1x32x512xf32, {}> loc(#loc9)
132         } {tiling_level = "L1_tiling"} loc(#loc9)
133         scf.forall.in_parallel {
134             tensor.parallel_insert_slice %29 into %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x32x512xf32, {}> into
tensor<1x512x2048xf32, {}> loc(#loc9)
135             } loc(#loc9)
136             } {tiling_level = "core_sharding"} loc(#loc9)
137             %inserted_slice = tensor.insert_slice %28 into %arg6[%arg1,
%arg3, %arg5] [1, 512, 2048] [1, 1, 1] : tensor<1x512x2048xf32, {}> into
tensor<1x1024x2048xf32, {}> loc(#loc9)
138             scf.yield %inserted_slice : tensor<1x1024x2048xf32, {}>
loc(#loc9)
139             } {tiling_level = "L2_tiling"} loc(#loc9)
140             scf.yield %25 : tensor<1x1024x2048xf32, {}> loc(#loc9)
141             } {tiling_level = "L2_tiling"} loc(#loc9)
142             scf.yield %24 : tensor<1x1024x2048xf32, {}> loc(#loc9)
143             } {tiling_level = "L2_tiling"} loc(#loc9)
144             %23 = "top.tileOutput"(%22) : (tensor<1x1024x2048xf32, {}>) ->
tensor<1x1024x2048xf32> loc(#loc8)

```

```

145         top.return %23 : tensor<1x1024x2048xf32> loc(#loc8)
146     } : tensor<1x1024x2048xf32> loc(#loc8)
147     %14 = top.tile {
148         %18 = "top.tileInput"(%12) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc6)
149         %19 = "top.tileInput"(%13) : (tensor<1x1024x2048xf32>) ->
tensor<1x1024x2048xf32, {}> loc(#loc6)
150         %20 = "top.tileInput"(%7) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc6)
151         %21 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32,
{}> loc(#loc6)
152         %22 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc6)
153         %23 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc11)
154         %24 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc12)
155         %25 = tensor.empty() : tensor<1x2048x2048xf32, {}> loc(#loc10)
156         %c0 = arith.constant 0 : index loc(#loc10)
157         %c1 = arith.constant 1 : index loc(#loc10)
158         %c0_0 = arith.constant 0 : index loc(#loc10)
159         %c1_1 = arith.constant 1 : index loc(#loc10)
160         %c0_2 = arith.constant 0 : index loc(#loc10)
161         %c1_3 = arith.constant 1 : index loc(#loc10)
162         %c1_4 = arith.constant 1 : index loc(#loc10)
163         %c2048 = arith.constant 2048 : index loc(#loc10)
164         %c1_5 = arith.constant 1 : index loc(#loc10)
165         %c2048_6 = arith.constant 2048 : index loc(#loc10)
166         %26 = scf.for %arg1 = %c0_2 to %c1_4 step %c1_5 iter_args(%arg2 = %25)
-> (tensor<1x2048x2048xf32, {}>) {
167             %28 = scf.for %arg3 = %c0_2 to %c2048 step %c2048_6 iter_args(%arg4
= %arg2) -> (tensor<1x2048x2048xf32, {}>) {
168                 %29 = "top.tileInput"(%12) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc6)
169                 %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
2048, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to tensor<1x2048x1024xf32,
{}> loc(#loc6)
170                 %30 = "top.tileInput"(%13) : (tensor<1x1024x2048xf32>) ->
tensor<1x1024x2048xf32, {}> loc(#loc6)
171                 %extracted_slice_7 = tensor.extract_slice %19[%arg1, 0, 0] [1,
1024, 2048] [1, 1, 1] : tensor<1x1024x2048xf32, {}> to tensor<1x1024x2048xf32,
{}> loc(#loc6)
172                 %extracted_slice_8 = tensor.extract_slice %22[%arg1, %arg3, 0] [1,
2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32> to tensor<1x2048x2048xf32>
loc(#loc6)
173                 %31 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) ->
tensor<1x1x1xf32, {}> loc(#loc6)
174                 %extracted_slice_9 = tensor.extract_slice %21[%arg1, 0, 0] [1, 1,
1] [1, 1, 1] : tensor<1x1x1xf32, {}> to tensor<1x1x1xf32, {}> loc(#loc11)

```

```

175         %extracted_slice_10 = tensor.extract_slice %23[%arg1, %arg3, 0]
[1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32> to tensor<1x2048x2048xf32>
loc(#loc11)
176         %32 = "top.tileInput"(%7) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc6)
177         %extracted_slice_11 = tensor.extract_slice %20[%arg1, %arg3, 0]
[1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> to
tensor<1x2048x2048xf32, {}> loc(#loc12)
178         %extracted_slice_12 = tensor.extract_slice %24[%arg1, %arg3, 0]
[1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32> to tensor<1x2048x2048xf32>
loc(#loc12)
179         %extracted_slice_13 = tensor.extract_slice %arg4[%arg1, %arg3,
%c0_2] [1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> to
tensor<1x2048x2048xf32, {}> loc(#loc10)
180         %c0_14 = arith.constant 0 : index loc(#loc10)
181         %c1_15 = arith.constant 1 : index loc(#loc10)
182         %c0_16 = arith.constant 0 : index loc(#loc10)
183         %c1_17 = arith.constant 1 : index loc(#loc10)
184         %c0_18 = arith.constant 0 : index loc(#loc10)
185         %c1_19 = arith.constant 1 : index loc(#loc10)
186         %33 = scf.forall (%arg5, %arg6) = (%c0_18, %c0_18) to (1, 2048)
step (1, 32) shared_outs(%arg7 = %extracted_slice_13) ->
(tensor<1x2048x2048xf32, {}>) {
187         %extracted_slice_20 = tensor.extract_slice
%extracted_slice[%arg5, %arg6, 0] [1, 32, 1024] [1, 1, 1] :
tensor<1x2048x1024xf32, {}> to tensor<1x32x1024xf32, {}> loc(#loc6)
188         %extracted_slice_21 = tensor.extract_slice
%extracted_slice_7[%arg5, 0, 0] [1, 1024, 2048] [1, 1, 1] :
tensor<1x1024x2048xf32, {}> to tensor<1x1024x2048xf32, {}> loc(#loc6)
189         %extracted_slice_22 = tensor.extract_slice
%extracted_slice_8[%arg5, %arg6, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x2048x2048xf32> to tensor<1x32x2048xf32> loc(#loc6)
190         %extracted_slice_23 = tensor.extract_slice
%extracted_slice_9[%arg5, 0, 0] [1, 1, 1] [1, 1, 1] : tensor<1x1x1xf32, {}> to
tensor<1x1x1xf32, {}> loc(#loc11)
191         %extracted_slice_24 = tensor.extract_slice
%extracted_slice_10[%arg5, %arg6, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x2048x2048xf32> to tensor<1x32x2048xf32> loc(#loc11)
192         %extracted_slice_25 = tensor.extract_slice
%extracted_slice_11[%arg5, %arg6, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x2048x2048xf32, {}> to tensor<1x32x2048xf32, {}> loc(#loc12)
193         %extracted_slice_26 = tensor.extract_slice
%extracted_slice_12[%arg5, %arg6, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x2048x2048xf32> to tensor<1x32x2048xf32> loc(#loc12)
194         %extracted_slice_27 = tensor.extract_slice %arg7[%arg5, %arg6,
%c0_18] [1, 32, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> to
tensor<1x32x2048xf32, {}> loc(#loc10)

```

```

195         %c0_28 = arith.constant 0 : index loc(#loc10)
196         %c1_29 = arith.constant 1 : index loc(#loc10)
197         %c0_30 = arith.constant 0 : index loc(#loc10)
198         %c1_31 = arith.constant 1 : index loc(#loc10)
199         %c1_32 = arith.constant 1 : index loc(#loc10)
200         %c32 = arith.constant 32 : index loc(#loc10)
201         %c1_33 = arith.constant 1 : index loc(#loc10)
202         %c32_34 = arith.constant 32 : index loc(#loc10)
203         %34 = scf.for %arg8 = %c0_30 to %c1_32 step %c1_33
iter_args(%arg9 = %extracted_slice_27) -> (tensor<1x32x2048xf32, {}>) {
204             %35 = scf.for %arg10 = %c0_30 to %c32 step %c32_34
iter_args(%arg11 = %arg9) -> (tensor<1x32x2048xf32, {}>) {
205                 %extracted_slice_35 = tensor.extract_slice
%extracted_slice_20[%arg8, %arg10, 0] [1, 32, 1024] [1, 1, 1] :
tensor<1x32x1024xf32, {}> to tensor<1x32x1024xf32, {}> loc(#loc6)
206                 %extracted_slice_36 = tensor.extract_slice
%extracted_slice_21[%arg8, 0, 0] [1, 1024, 2048] [1, 1, 1] :
tensor<1x1024x2048xf32, {}> to tensor<1x1024x2048xf32, {}> loc(#loc6)
207                 %extracted_slice_37 = tensor.extract_slice
%extracted_slice_22[%arg8, %arg10, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32> to tensor<1x32x2048xf32> loc(#loc6)
208                 %36 = linalg.batch_matmul ins(%extracted_slice_35,
%extracted_slice_36 : tensor<1x32x1024xf32, {}>, tensor<1x1024x2048xf32, {}>)
outs(%extracted_slice_37 : tensor<1x32x2048xf32>) -> tensor<1x32x2048xf32>
loc(#loc6)
209                 %extracted_slice_38 = tensor.extract_slice
%extracted_slice_23[%arg8, 0, 0] [1, 1, 1] [1, 1, 1] : tensor<1x1x1xf32, {}>
to tensor<1x1x1xf32, {}> loc(#loc11)
210                 %extracted_slice_39 = tensor.extract_slice
%extracted_slice_24[%arg8, %arg10, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32> to tensor<1x32x2048xf32> loc(#loc11)
211                 %37 = linalg_ext.mul ins(%36, %extracted_slice_38 :
tensor<1x32x2048xf32>, tensor<1x1x1xf32, {}>) outs(%extracted_slice_39 :
tensor<1x32x2048xf32>) -> tensor<1x32x2048xf32> loc(#loc11)
212                 %extracted_slice_40 = tensor.extract_slice
%extracted_slice_25[%arg8, %arg10, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32, {}> to tensor<1x32x2048xf32, {}> loc(#loc12)
213                 %extracted_slice_41 = tensor.extract_slice
%extracted_slice_26[%arg8, %arg10, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32> to tensor<1x32x2048xf32> loc(#loc12)
214                 %38 = linalg.add ins(%37, %extracted_slice_40 :
tensor<1x32x2048xf32>, tensor<1x32x2048xf32, {}>) outs(%extracted_slice_41 :
tensor<1x32x2048xf32>) -> tensor<1x32x2048xf32> loc(#loc12)
215                 %extracted_slice_42 = tensor.extract_slice %arg11[%arg8,
%arg10, %c0_30] [1, 32, 2048] [1, 1, 1] : tensor<1x32x2048xf32, {}> to
tensor<1x32x2048xf32, {}> loc(#loc10)

```

```

216         %39 = linalg.softmax {is_root = true} dimension(2) ins(%38 :
tensor<1x32x2048xf32>) outs(%extracted_slice_42 : tensor<1x32x2048xf32, {}>) -
> tensor<1x32x2048xf32, {}> loc(#loc10)
217         %inserted_slice_43 = tensor.insert_slice %39 into
%arg11[%arg8, %arg10, %c0_30] [1, 32, 2048] [1, 1, 1] : tensor<1x32x2048xf32,
{}> into tensor<1x32x2048xf32, {}> loc(#loc10)
218         scf.yield %inserted_slice_43 : tensor<1x32x2048xf32, {}>
loc(#loc10)
219         } {tiling_level = "L1_tiling"} loc(#loc10)
220         scf.yield %35 : tensor<1x32x2048xf32, {}> loc(#loc10)
221         } {tiling_level = "L1_tiling"} loc(#loc10)
222         scf.forall.in_parallel {
223             tensor.parallel_insert_slice %34 into %arg7[%arg5, %arg6,
%c0_18] [1, 32, 2048] [1, 1, 1] : tensor<1x32x2048xf32, {}> into
tensor<1x2048x2048xf32, {}> loc(#loc10)
224             } loc(#loc10)
225             } {tiling_level = "core_sharding"} loc(#loc10)
226             %inserted_slice = tensor.insert_slice %33 into %arg4[%arg1, %arg3,
%c0_2] [1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> into
tensor<1x2048x2048xf32, {}> loc(#loc10)
227             scf.yield %inserted_slice : tensor<1x2048x2048xf32, {}> loc(#loc10)
228             } {tiling_level = "L2_tiling"} loc(#loc10)
229             scf.yield %28 : tensor<1x2048x2048xf32, {}> loc(#loc10)
230             } {tiling_level = "L2_tiling"} loc(#loc10)
231             %27 = "top.tileOutput"(%26) : (tensor<1x2048x2048xf32, {}>) ->
tensor<1x2048x2048xf32> loc(#loc6)
232             top.return %27 : tensor<1x2048x2048xf32> loc(#loc6)
233             } : tensor<1x2048x2048xf32> loc(#loc6)
234             %15 = top.tile {
235                 %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc13)
236                 %19 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc13)
237                 %20 = tensor.empty() : tensor<1x2048x1024xf32, {}> loc(#loc13)
238                 %c0 = arith.constant 0 : index loc(#loc13)
239                 %c0_0 = arith.constant 0 : index loc(#loc13)
240                 %c0_1 = arith.constant 0 : index loc(#loc13)
241                 %c1 = arith.constant 1 : index loc(#loc13)
242                 %c2048 = arith.constant 2048 : index loc(#loc13)
243                 %c1024 = arith.constant 1024 : index loc(#loc13)
244                 %c1_2 = arith.constant 1 : index loc(#loc13)
245                 %c1024_3 = arith.constant 1024 : index loc(#loc13)
246                 %c1024_4 = arith.constant 1024 : index loc(#loc13)
247                 %21 = scf.for %arg1 = %c0 to %c1 step %c1_2 iter_args(%arg2 = %20) ->
(tensor<1x2048x1024xf32, {}>) {
248                     %23 = scf.for %arg3 = %c0_0 to %c2048 step %c1024_3 iter_args(%arg4
= %arg2) -> (tensor<1x2048x1024xf32, {}>) {

```

```

249         %24 = scf.for %arg5 = %c0_1 to %c1024 step %c1024_4
iter_args(%arg6 = %arg4) -> (tensor<1x2048x1024xf32, {}>) {
250         %25 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc13)
251         %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
1024, 512] [1, 1, 1] : tensor<1x2048x512xf32, {}> to tensor<1x1024x512xf32,
{}> loc(#loc13)
252         %26 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc13)
253         %extracted_slice_5 = tensor.extract_slice %19[%arg1, 0, %arg5]
[1, 512, 1024] [1, 1, 1] : tensor<1x512x1024xf32, {}> to
tensor<1x512x1024xf32, {}> loc(#loc13)
254         %extracted_slice_6 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to
tensor<1x1024x1024xf32, {}> loc(#loc13)
255         %27 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 1024,
1024) step (1, 32, 512) shared_outs(%arg10 = %extracted_slice_6) ->
(tensor<1x1024x1024xf32, {}>) {
256         %extracted_slice_7 = tensor.extract_slice
%extracted_slice[%arg7, %arg8, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x1024x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc13)
257         %extracted_slice_8 = tensor.extract_slice
%extracted_slice_5[%arg7, 0, %arg9] [1, 512, 512] [1, 1, 1] :
tensor<1x512x1024xf32, {}> to tensor<1x512x512xf32, {}> loc(#loc13)
258         %extracted_slice_9 = tensor.extract_slice %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x1024x1024xf32, {}> to
tensor<1x32x512xf32, {}> loc(#loc13)
259         %c0_10 = arith.constant 0 : index loc(#loc13)
260         %c0_11 = arith.constant 0 : index loc(#loc13)
261         %c0_12 = arith.constant 0 : index loc(#loc13)
262         %c1_13 = arith.constant 1 : index loc(#loc13)
263         %c32 = arith.constant 32 : index loc(#loc13)
264         %c512 = arith.constant 512 : index loc(#loc13)
265         %c1_14 = arith.constant 1 : index loc(#loc13)
266         %c32_15 = arith.constant 32 : index loc(#loc13)
267         %c32_16 = arith.constant 32 : index loc(#loc13)
268         %28 = scf.for %arg11 = %c0_10 to %c1_13 step %c1_14
iter_args(%arg12 = %extracted_slice_9) -> (tensor<1x32x512xf32, {}>) {
269         %29 = scf.for %arg13 = %c0_11 to %c32 step %c32_15
iter_args(%arg14 = %arg12) -> (tensor<1x32x512xf32, {}>) {
270         %30 = scf.for %arg15 = %c0_12 to %c512 step %c32_16
iter_args(%arg16 = %arg14) -> (tensor<1x32x512xf32, {}>) {
271         %extracted_slice_17 = tensor.extract_slice
%extracted_slice_7[%arg11, %arg13, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x32x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc13)
272         %extracted_slice_18 = tensor.extract_slice
%extracted_slice_8[%arg11, 0, %arg15] [1, 512, 32] [1, 1, 1] :

```

```

tensor<1x512x512xf32, {}> to tensor<1x512x32xf32, {}> loc(#loc13)
273         %extracted_slice_19 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x512xf32,
{}> to tensor<1x32x32xf32, {}> loc(#loc13)
274         %31 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_17, %extracted_slice_18 : tensor<1x32x512xf32, {}>,
tensor<1x512x32xf32, {}>) outs(%extracted_slice_19 : tensor<1x32x32xf32, {}>) -
> tensor<1x32x32xf32, {}> loc(#loc13)
275         %inserted_slice_20 = tensor.insert_slice %31 into
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x32xf32, {}>
into tensor<1x32x512xf32, {}> loc(#loc13)
276         scf.yield %inserted_slice_20 : tensor<1x32x512xf32, {}>
loc(#loc13)
277         } {tiling_level = "L1_tiling"} loc(#loc13)
278         scf.yield %30 : tensor<1x32x512xf32, {}> loc(#loc13)
279         } {tiling_level = "L1_tiling"} loc(#loc13)
280         scf.yield %29 : tensor<1x32x512xf32, {}> loc(#loc13)
281         } {tiling_level = "L1_tiling"} loc(#loc13)
282         scf.forall.in_parallel {
283             tensor.parallel_insert_slice %28 into %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x32x512xf32, {}> into
tensor<1x1024x1024xf32, {}> loc(#loc13)
284             } loc(#loc13)
285             } {tiling_level = "core_sharding"} loc(#loc13)
286             %inserted_slice = tensor.insert_slice %27 into %arg6[%arg1,
%arg3, %arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x1024x1024xf32, {}> into
tensor<1x2048x1024xf32, {}> loc(#loc13)
287             scf.yield %inserted_slice : tensor<1x2048x1024xf32, {}>
loc(#loc13)
288             } {tiling_level = "L2_tiling"} loc(#loc13)
289             scf.yield %24 : tensor<1x2048x1024xf32, {}> loc(#loc13)
290             } {tiling_level = "L2_tiling"} loc(#loc13)
291             scf.yield %23 : tensor<1x2048x1024xf32, {}> loc(#loc13)
292             } {tiling_level = "L2_tiling"} loc(#loc13)
293             %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {}>) ->
tensor<1x2048x1024xf32> loc(#loc13)
294             top.return %22 : tensor<1x2048x1024xf32> loc(#loc13)
295             } : tensor<1x2048x1024xf32> loc(#loc13)
296             %16 = top.tile {
297                 %18 = "top.tileInput"(%15) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc14)
298                 %19 = "top.tileInput"(%14) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc14)
299                 %20 = tensor.empty() : tensor<1x2048x1024xf32, {}> loc(#loc14)
300                 %c0 = arith.constant 0 : index loc(#loc14)
301                 %c0_0 = arith.constant 0 : index loc(#loc14)
302                 %c0_1 = arith.constant 0 : index loc(#loc14)

```

```

303         %c1 = arith.constant 1 : index loc(#loc14)
304         %c2048 = arith.constant 2048 : index loc(#loc14)
305         %c1024 = arith.constant 1024 : index loc(#loc14)
306         %c1_2 = arith.constant 1 : index loc(#loc14)
307         %c1024_3 = arith.constant 1024 : index loc(#loc14)
308         %c1024_4 = arith.constant 1024 : index loc(#loc14)
309         %21 = scf.for %arg1 = %c0 to %c1 step %c1_2 iter_args(%arg2 = %20) ->
(tensor<1x2048x1024xf32, {}>) {
310             %23 = scf.for %arg3 = %c0_0 to %c2048 step %c1024_3 iter_args(%arg4
= %arg2) -> (tensor<1x2048x1024xf32, {}>) {
311                 %24 = scf.for %arg5 = %c0_1 to %c1024 step %c1024_4
iter_args(%arg6 = %arg4) -> (tensor<1x2048x1024xf32, {}>) {
312                     %25 = "top.tileInput"(%14) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc14)
313                     %extracted_slice = tensor.extract_slice %19[%arg1, %arg3, 0] [1,
1024, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> to tensor<1x1024x2048xf32,
{}> loc(#loc14)
314                     %26 = "top.tileInput"(%15) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc14)
315                     %extracted_slice_5 = tensor.extract_slice %18[%arg1, 0, %arg5]
[1, 2048, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to
tensor<1x2048x1024xf32, {}> loc(#loc14)
316                     %extracted_slice_6 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to
tensor<1x1024x1024xf32, {}> loc(#loc14)
317                     %27 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 1024,
1024) step (1, 32, 512) shared_outs(%arg10 = %extracted_slice_6) ->
(tensor<1x1024x1024xf32, {}>) {
318                         %extracted_slice_7 = tensor.extract_slice
%extracted_slice[%arg7, %arg8, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x1024x2048xf32, {}> to tensor<1x32x2048xf32, {}> loc(#loc14)
319                         %extracted_slice_8 = tensor.extract_slice
%extracted_slice_5[%arg7, 0, %arg9] [1, 2048, 512] [1, 1, 1] :
tensor<1x2048x1024xf32, {}> to tensor<1x2048x512xf32, {}> loc(#loc14)
320                         %extracted_slice_9 = tensor.extract_slice %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x1024x1024xf32, {}> to
tensor<1x32x512xf32, {}> loc(#loc14)
321                         %c0_10 = arith.constant 0 : index loc(#loc14)
322                         %c0_11 = arith.constant 0 : index loc(#loc14)
323                         %c0_12 = arith.constant 0 : index loc(#loc14)
324                         %c1_13 = arith.constant 1 : index loc(#loc14)
325                         %c32 = arith.constant 32 : index loc(#loc14)
326                         %c512 = arith.constant 512 : index loc(#loc14)
327                         %c1_14 = arith.constant 1 : index loc(#loc14)
328                         %c32_15 = arith.constant 32 : index loc(#loc14)
329                         %c32_16 = arith.constant 32 : index loc(#loc14)

```

```

330         %28 = scf.for %arg11 = %c0_10 to %c1_13 step %c1_14
iter_args(%arg12 = %extracted_slice_9) -> (tensor<1x32x512xf32, {}>) {
331         %29 = scf.for %arg13 = %c0_11 to %c32 step %c32_15
iter_args(%arg14 = %arg12) -> (tensor<1x32x512xf32, {}>) {
332         %30 = scf.for %arg15 = %c0_12 to %c512 step %c32_16
iter_args(%arg16 = %arg14) -> (tensor<1x32x512xf32, {}>) {
333         %extracted_slice_17 = tensor.extract_slice
%extracted_slice_7[%arg11, %arg13, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32, {}> to tensor<1x32x2048xf32, {}> loc(#loc14)
334         %extracted_slice_18 = tensor.extract_slice
%extracted_slice_8[%arg11, 0, %arg15] [1, 2048, 32] [1, 1, 1] :
tensor<1x2048x512xf32, {}> to tensor<1x2048x32xf32, {}> loc(#loc14)
335         %extracted_slice_19 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x512xf32,
{}> to tensor<1x32x32xf32, {}> loc(#loc14)
336         %31 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_17, %extracted_slice_18 : tensor<1x32x2048xf32, {}>,
tensor<1x2048x32xf32, {}>) outs(%extracted_slice_19 : tensor<1x32x32xf32, {}>)
-> tensor<1x32x32xf32, {}> loc(#loc14)
337         %inserted_slice_20 = tensor.insert_slice %31 into
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x32xf32, {}>
into tensor<1x32x512xf32, {}> loc(#loc14)
338         scf.yield %inserted_slice_20 : tensor<1x32x512xf32, {}>
loc(#loc14)
339         } {tiling_level = "L1_tiling"} loc(#loc14)
340         scf.yield %30 : tensor<1x32x512xf32, {}> loc(#loc14)
341         } {tiling_level = "L1_tiling"} loc(#loc14)
342         scf.yield %29 : tensor<1x32x512xf32, {}> loc(#loc14)
343         } {tiling_level = "L1_tiling"} loc(#loc14)
344         scf.forall.in_parallel {
345         tensor.parallel_insert_slice %28 into %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x32x512xf32, {}> into
tensor<1x1024x1024xf32, {}> loc(#loc14)
346         } loc(#loc14)
347         } {tiling_level = "core_sharding"} loc(#loc14)
348         %inserted_slice = tensor.insert_slice %27 into %arg6[%arg1,
%arg3, %arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x1024x1024xf32, {}> into
tensor<1x2048x1024xf32, {}> loc(#loc14)
349         scf.yield %inserted_slice : tensor<1x2048x1024xf32, {}>
loc(#loc14)
350         } {tiling_level = "L2_tiling"} loc(#loc14)
351         scf.yield %24 : tensor<1x2048x1024xf32, {}> loc(#loc14)
352         } {tiling_level = "L2_tiling"} loc(#loc14)
353         scf.yield %23 : tensor<1x2048x1024xf32, {}> loc(#loc14)
354         } {tiling_level = "L2_tiling"} loc(#loc14)
355         %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {}>) ->
tensor<1x2048x1024xf32> loc(#loc14)

```

```

356         top.return %22 : tensor<1x2048x1024xf32> loc(#loc14)
357     } : tensor<1x2048x1024xf32> loc(#loc14)
358     %17 = "top.fusionOutput"(%16) : (tensor<1x2048x1024xf32>) ->
        tensor<1x2048x1024xf32> loc(#loc6)
359     top.return %17 : tensor<1x2048x1024xf32> loc(#loc6)
360 } : tensor<1x2048x1024xf32> loc(#loc6)
361 return %5 : tensor<1x2048x1024xf32> loc(#loc15)
362 } loc(#loc)
363 } loc(#loc)
364
365

```

2.6 Schedule优化

2.6.1 流水编排

代码块

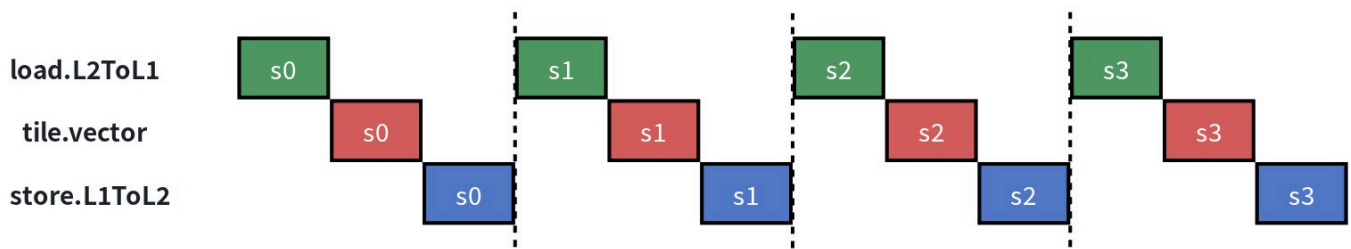
```

1  %l2_input = ...
2  %l2_output = holo.alloc.l2()
3  scf.for %c = 0 to 128 step 32 {
4      %l1_input = tile.dma.load(%l2_input, %c, 32)
5      %l1_output = tile.relu(%l1_input)
6      tile.dma.store(%l1_output, %l2_output)
7  }

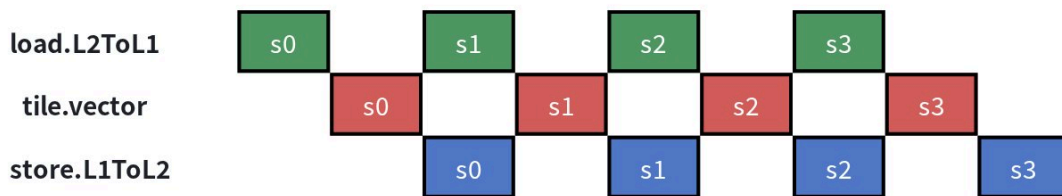
```

IR分析后, 分配2块L1 memory buffer, 对应的执行流如下

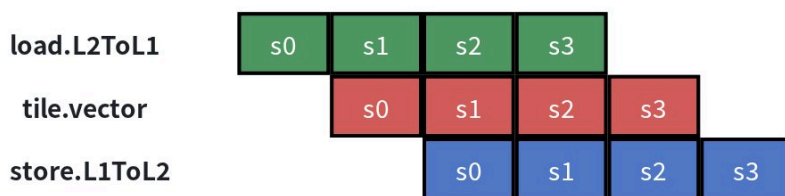
循环不展开, barrier同步



循环不展开, flag同步



循环部分展开, ping-pong流水



代码块

```
1  %l2_input = ...
2  %l2_output = holo.alloc.l2()
3  scf.for %c = 0 to 128 step 64 {
4      %l1_input_ping = tile.dma.load(%l2_input, %c, 32)
5      %l1_output_ping = tile.relu(%l1_input_ping)
6      tile.dma.store(%l1_output_ping, %l2_output)
7
8      %l1_input_pong = tile.dma.load(%l2_input, %c + 32, 32)
9      %l1_output_pong = tile.relu(%l1_input_pong)
```

```
10     tile.dma.store(%l1_output_pong, %l2_output)
11 }
```

? 是否有更好的方式表达ping-pong buffer

2.7 Memory Planning

Memory Space	Memory Scope
DDR	Input/Output, Weight/Param, Code, Intermedia
Shared Memory	
L1	
Register File	General Regs, Tile Regs, Acc Regs, Predicate Regs

2.7.1 启发式

生命周期分析 + First-Fit, Best-Fit等, 速度快, 性能接近求解器.

1. L1分配例子

由于L1的使用被限制在subgraph之内(包括subgraph input load, output store), 而同一个tile内, subgraph的执行是

串行执行的, 所以不同subgraph内的L1 tensor不存在生命周期交叠, 因此在分配L1时, 可以将分析范围限定在subgraph内(不考虑L1 persistent mem)。

1. 收集所有mem-space是L1的tensor, 并分析其生命周期
2. 对于任意tensor x , 它的生命周期是 $[start_x, end_x]$, 若 $end_max = \max(end_x)$, 则所有tensor的生命周期均在 $[0, end_max]$
3. 对于时刻 t , 收集所有liveness start = t 的tensor, 并按照tensor size或者liveness长短进行排序, 该序列记为 $t_tensors$;

4. 当 $t = 0$ 时, L1未被占用, 则按照 $t_tensors$ 依次分配, 对于tensor i , 它分配的区间是 $[start_i, end_i]$, 任意两个tensor区间都是不重叠, 并将分配情况记录下 $l1_tensors$
5. 在时刻 t , 若对于 $l1_tensors$ 中的tensor i , 它的liveness end 小于 t , 则将其从 $l1_tensors$ 中删除
6. 在时刻 t , 对于 $t_tensors$ 中的tensor i , 找到L1中第一块不被 $l1_tensors$ 中tensor占用, 且满足大小需求的区间, 分配给tensor i , 直到完成所有tensor的分配
7. 当 $t > end_max$, 结束分配

? 是否考虑地址冲突, 可以给sharding/tiling兜底, 但引入假依赖, 增加同步开销

2.7.2 整数线性规划求解器

生命周期重叠分析 + IPL solver, 性能高, 求解速度慢.

2.8 同步

2.8.1 同步类型

同步类型			是否需要软件同步	分析范围
数据依赖RAW	Tile内	Mini & Cisc Core指令	不需要, lsu load/store和mini scalar load/store指令除外	只涉及L1 tensor相关的依赖关系, 分析范围限定在tile-graph
		(Mini & Cisc Core) 与 TDMA指令	需要, 使用 set_flag/wait_flag 或 pipe.barrier	
	Tile间	TDMA指令或其它可访问DDR, L2 shared的指令	需要, 使用 set_event/wait_event 或tile.barrier	涉及DDR/L2 tensor相关的依赖关系, 分析范围是tile-graph以外的代码
	chip间	未知	需要, 是否依赖推理引擎/runtime调度	涉及DDR tensor相关的依赖关系

资源冲突	计算资源: Tensor Core, Vector Core, LSU, TDMA	不需要, 硬件指令队列是保序的	
	内存资源: DDR, L2/L1, Register	需要	WAR, WAW
	其他资源: tile内同步用的flag_id(64个), tile间同步用的event_id(64个)	需要	

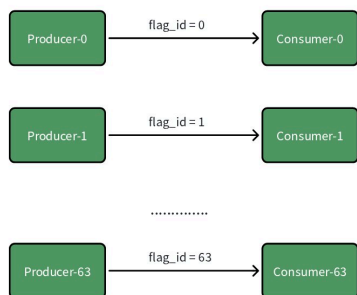
scope (指令或代码最终执行单元)	硬件已有能力	硬件缺失能力	软件使用场景 举例(极端或死锁场景)
Risc-v Core	risc-v同步指令, 用Rsync-set指代		
Mini Core	- Mini Core代码间通过fence同步 - Mini Core代码与Cisc Core指令间通过fence同步 - Cisc Core指令间的同步分成如下两种情况: - mma和vector指令的同步硬件负责 - lsu.load和lsu.store指令的同步由软件负责	缺少一条用于lsu.load和lsu.store间同步的fence指令	
TDMA.load	指令队列内保序		
TDMA.store	指令队列内保序		
Acc	指令队列内保序		
Tile内	- Risc-v Core代码间通过Rsync-set同步 - Risc-v Core代码与TDMA.load/TDMA.store/Acc/Mini Core间通过Rsync-set同步 - TDMA.load/TDMA.store/Acc/Mini Core间通过set_flag/wait_flag 或 pipe.barrier同步		
Tile间	tile与tile间使用set_event/wait_event, tile.barrier或Atomic同步		
Host与NPU间	mailbox或中断		

2.8.2 flag_id/event_id资源冲突

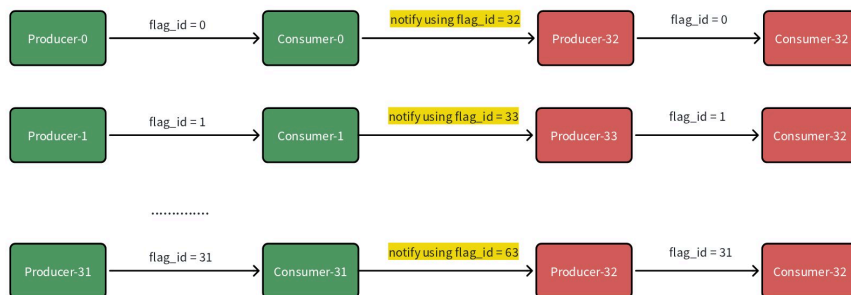
2.8.2.1 flag_id的使用方式

flag_id主要用于tile core内部指令间的同步。

{producer-64, Consumer-64}如何知道
{Producer-0, Consumer-0}是否已经完成,
使用flag_id = 0



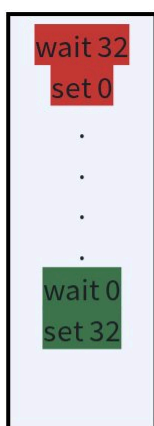
flag_id一半用于真实同步,一半用于flag_id资源冲突。
每个flag_id都需要一个配对的flag_id,用于通知下游该flag_id可复用



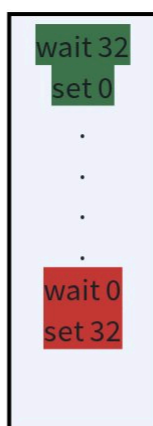
1. 如下使用方式, 有死锁风险

多个queue使用同一个flag_id

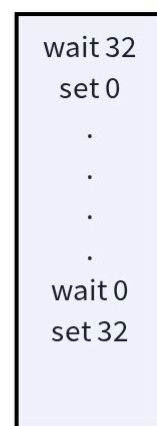
CISC



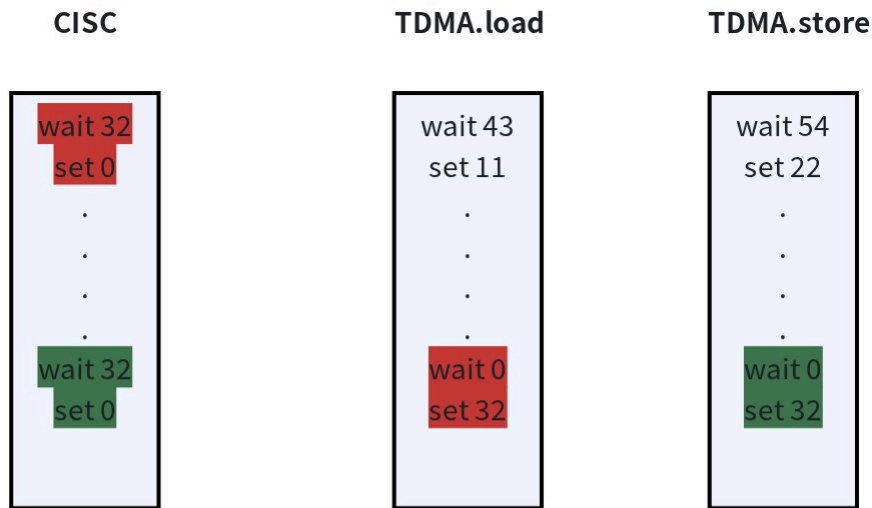
TDMA.load



TDMA.store



2. 如果限制每个queue能够set的flag_id范围, 比如CISC使用[0, 10], TDMA.load使用[11, 21], TDMA.store使用[22, 31], 此时没有死锁风险, 但存在消费者消费错误的风险



如果按照如下{src stream, dst stream}分配, 对于任意一个flag_id, 只有一个queue会对它set, 一个queue对它wait, 此时没有2存在的风险

代码块

```

1  // {src, dst}
2  {CISC, TDMA.load}           // [0, 4]
3  {CISC, TDMA.store}         // [5, 9]
4  {TDMA.load, CISC}          // [10, 14]
5  {TDMA.load, TDMA.store}    // [15, 19]
6  {TDMA.store, CISC}         // [20, 25]
7  {TDMA.store, TDMA.load}    // [26, 31]
8
9  // 对于上面32个flag_id的使用采用如下方式, 假设flag_id = i
10 // 在程序开头如下添加flag_id初始化代码
11 set_all_flag_id(1)
12 barrier()
13
14 // producer                      // consumer
15 wait(i+32)                        wait(i)
16 set(i)                            set(i+32)
17
18 // 需求
19 根据负载评估下, 等flag_id的场景是否频繁, 增加flag_id的数目

```

2.8.2.2 event_id的使用方式

event_id用于tile core之间的事件同步.

event与flag同步有以下两点不同:

1. 当在tile core A上执行set_event时, tile core上的所有写内存操作对其它tile core都是可见的
2. 每个tile core都拥有64个event id

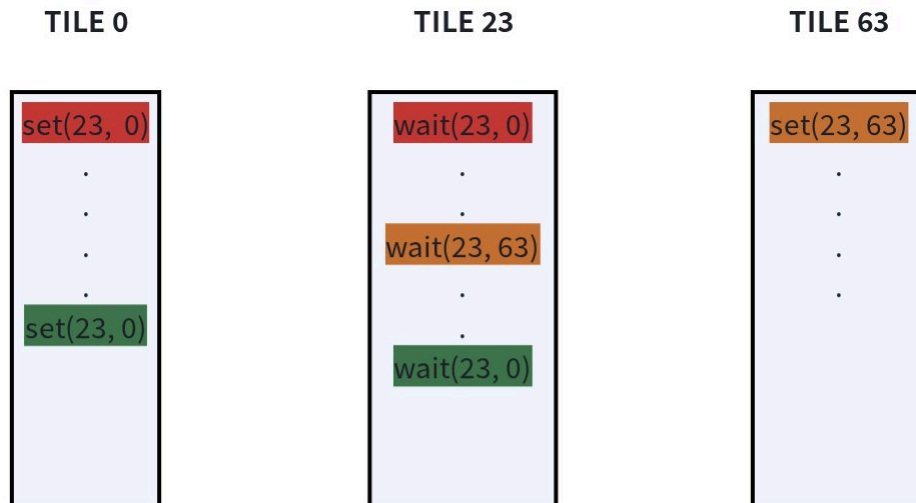
从producer tile core视角出发, 若它需要通知远端consumer tile core, 使用的event_id与producer tile core id一致

代码块

```
1  set_event(consumer-tile-core-id, event_id = producer-tile-core-id)
```

如下图

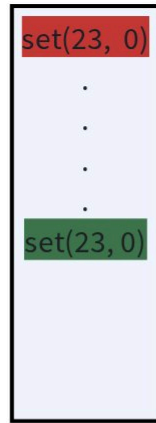
1. 由于不同的producer使用不同的event_id与consumer通信, 所以consumer消费不同的producer的event不会错乱
2. 如果tile 0连续向tile 23发送了两个event, 由于set_event/wait_event在tile内是顺序执行的, 所以consumer消费同一个producer的两个event不会错乱



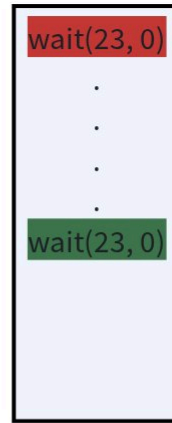
消费者错误消费

两次set在第一个wait前完成

TILE 0

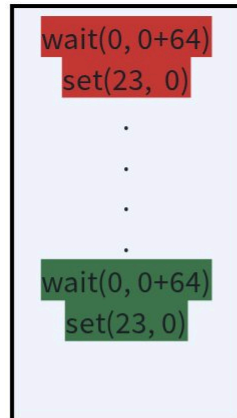


TILE 23

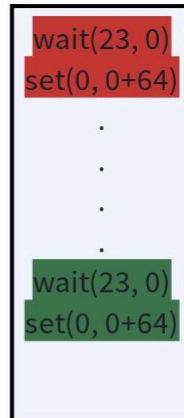


如果参照flag_id的使用方式, 将每个tile core的event id数目从64提升到128, 增加的64个event id用于前64个event id资源竞争时的同步, 即 $\text{event_id} = i + 64$ 用于 $\text{event_id} = i$ 的资源竞争同步.

TILE 0

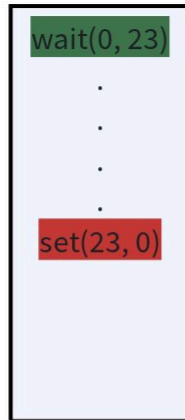


TILE 23

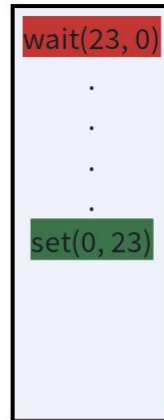


死锁场景

TILE 0



TILE 23



在SPMD场景下, 每个tile上的代码一致, 不会出现这种情况;

在Dataflow场景下, 两个stage之间通过ping-pong buffer交换数据

代码块

```
1  TILE 0
2  // wait ping consumer
3  wait(23, 23)
4  mma
5  // ping producer done
6  set(23, 0)
```

```
TILE 23
// wait ping producer
wait(0, 0)
softmax
// ping consumer done
set(0, 23)
```

结构上看上去一样, 但ping-pong buffer场景下, 第一笔是不用等待的, 所以不会死锁.

event_id的使用方式如下

代码块

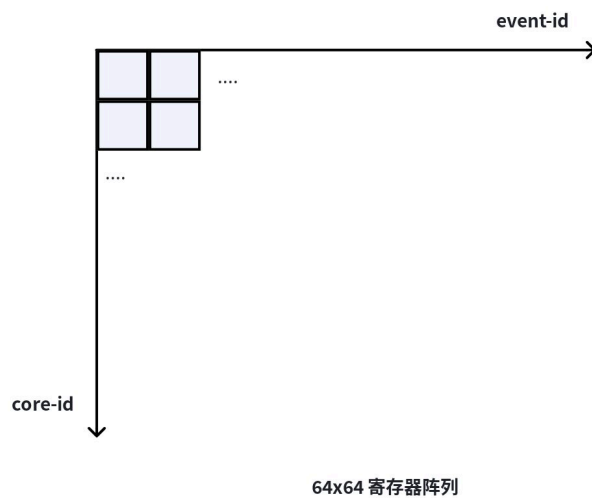
```
1  // 当两个tile core之间需要通信时, 假设producer core 和 consumer core
2  // 的core id分别是producer-tile-core-id, consumer-tile-core-id.
3
4  // 将每个tile core 128个event id分成[0, 63]和[64, 127]两个部分
5  // [0, 63]用于tile间的同步, [64, 127]用于event id资源竞争的同步
6  // 即将event_id = (i + 64)用于event_id = i的资源竞争的同步
7
8  // 在程序开头如下添加event_id初始化代码
9  set_all_event_id(1)
10 barrier()
11
12 // producer端采用如下指令配置
13 wait_event(producer-tile-core-id, event_id = producer-tile-core-id + 64)
14 set_event(consumer-tile-core-id, event_id = producer-tile-core-id)
```

```

15
16 // consumer端采用如下指令配置
17 wait_event(consumer-tile-core-id, event_id = producer-tile-core-id)
18 set_event(producer-tile-core-id, event_id = producer-tile-core-id + 64)
19
20 // 需求
21 每个tile的event_id至少增加到128

```

相当于每个{core a, core b}对间的通信使用唯一的event reg，同时搭配另外一个event reg保存它的使用状态.



2.8.3 场景

1. CICS指令同步

mini-core指令与cisc-core指令间是保序的？

lsu.store和lsu.load之间的同步？

代码块

```

1  mma acc0, t0, t1
2  lsu.store acc0, [rs1]
3  ...
4  lb rd0, [rs1] // mini-core
5  ...
6  lsu.load acc1, [rs1]
7
8
9  // 方案1

```

```
10 fence
11
12 // 硬件同步
13 lsu支持地址分析
14
```

2.9 算子库调用

代码块

```
1 // private函数，表示外部函数，该符号在链接时由外部提供
2 func.func private @holo_gemm(
3     %transA: i32, %transB: i32,
4     %M: i32, %N: i32, %K: i32,
5     %alpha: f32,
6     %A: memref<128x64xf32>, %lda: i32,
7     %B: memref<64x256xf32>, %ldb: i32,
8     %beta: f32,
9     %C: memref<128x256xf32>, %ldc: i32
10 ) -> i32
11
12
13 func.func @matmul(%A: memref<128x64xf32>, %B: memref<64x256xf32>, %C:
14     memref<128x256xf32>) {
15     %0 = arith.constant 0 : i32
16     %1 = arith.constant 1 : i32
17     %128 = arith.constant 128 : i32
18     %256 = arith.constant 256 : i32
19     %64 = arith.constant 64 : i32
20     %alpha = arith.constant 1.0 : f32
21     %beta = arith.constant 0.0 : f32
22
23     func.call @holo_gemm(
24         %0, %0, %128, %256, %64,
25         %alpha, %A, %128, %B, %64, %beta, %C, %128
26     ) : (!llvm.ptr, i32, i32, i32, i32, i32, i32, f32,
27         memref<128x64xf32>, i32, memref<64x256xf32>, i32, f32,
28         memref<128x256xf32>, i32) -> i32
29     return
30 }
```

3. 调研和实验

3.1 Linalg Dialect

优势: 统一的接口描述算子, 便于tiling和fusion

代码块

```
1  // 例子: conv
2  // strides = [1, 1]
3  // %input : tensor<1x3x32x32xf32> // NCHW
4  // %filter : tensor<16x3x3x3xf32> // FCHW (out_c, in_c, kh, kw)
5  // %output : tensor<1x16x30x30xf32> // N F OH OW (32-3+1=30)
6
7  #map_in = affine_map<(n, f, oh, ow, c, kh, kw) -> (n, c, oh + kh, ow + kw)>
8  #map_w = affine_map<(n, f, oh, ow, c, kh, kw) -> (f, c, kh, kw)>
9  #map_out = affine_map<(n, f, oh, ow, c, kh, kw) -> (n, f, oh, ow)>
10
11 %0 = linalg.generic {
12   indexing_maps = [#map_in, #map_w, #map_out],
13   iterator_types = [
14     "parallel", // n
15     "parallel", // f
16     "parallel", // oh
17     "parallel", // ow
18     "reduction", // c
19     "reduction", // kh
20     "reduction" // kw
21   ]
22 } ins(%input, %filter : tensor<1x3x32x32xf32>, tensor<16x3x3x3xf32>)
23   outs(%output : tensor<1x16x30x30xf32>) {
24   ^bb0(%x: f32, %w: f32, %acc: f32):
25     %mul = arith.mulf %x, %w : f32
26     %add = arith.addf %acc, %mul : f32
27     linalg.yield %add : f32
28 } -> tensor<1x16x30x30xf32>
29
30 要求:
31 1. 完美嵌套循环 (每个迭代空间维度对应一层 loop)
32 2. 每个operand有affine indexing_map
33 3. iterator类型是parallel/reduction
34 4. payload是纯标量计算(region里只有arith等)
```

payload使用标量描述，适合GPU simt编程模型; 与NPU的Tensorization下降需求不太匹配

代码块

```
1  for (i ...)
2    for (j ...)
3      for (k ...)
4        C[i][j] += A[i][k] * B[k][j];  // thread内的标量计算
5
6
7  for (i_tile ...)
8    for (j_tile ...)
9      for (k_tile ...)
10       mma(C_tile, A_tile, B_tile);  // 调用16x16x16矩阵块计算的硬件原语
```

存在一些op, 无法使用generic进行描述

代码块

```
1  // 采样算子: gather, scatter
2  scatter(input, indices, updates): input[indices[i, j, k]] = updates[i, j, k]
3  无法给出input/output的indexing_map
4
5  // 排序算子: sort, topk
6  1. 需要多次遍历
7  2. 无法给出输出的indexing_map
8
9  // 复合算子: softmax, attention
10 1. 不是一次遍历完成，需要拆分成多个linalg.generic描述
```

3.1.1 Linalg扩展

没有linalg.generic描述, 但实现了linalg tiling/fusion interface (<https://github.com/iree-org/iree/blob/1e43d5458e3342e7644bc3b9b4ef61c38a17da7a/compiler/src/iree/compiler/Dialect/LinalgExt/IR/LinalgExtOps.td#L47>)

3.1.2 linalg fuse

类型	说明	操作
Elementwise fusion	合成一个linalg.generic	

	入口: linalg::fuseElementwiseOps	argMap.compose(inversePermutation(producerResultMap)).compose(consumerMap)
Tile-and-Fuse	基于scf.for, 先切consumer, 再把producer算进同一套循环 scf::tileConsumerAndFuseProducersUsingSCF	

3.1.3 遗留问题

☐ 无法走linalg tile-and-fuse的算子, 需要有其它下降路线

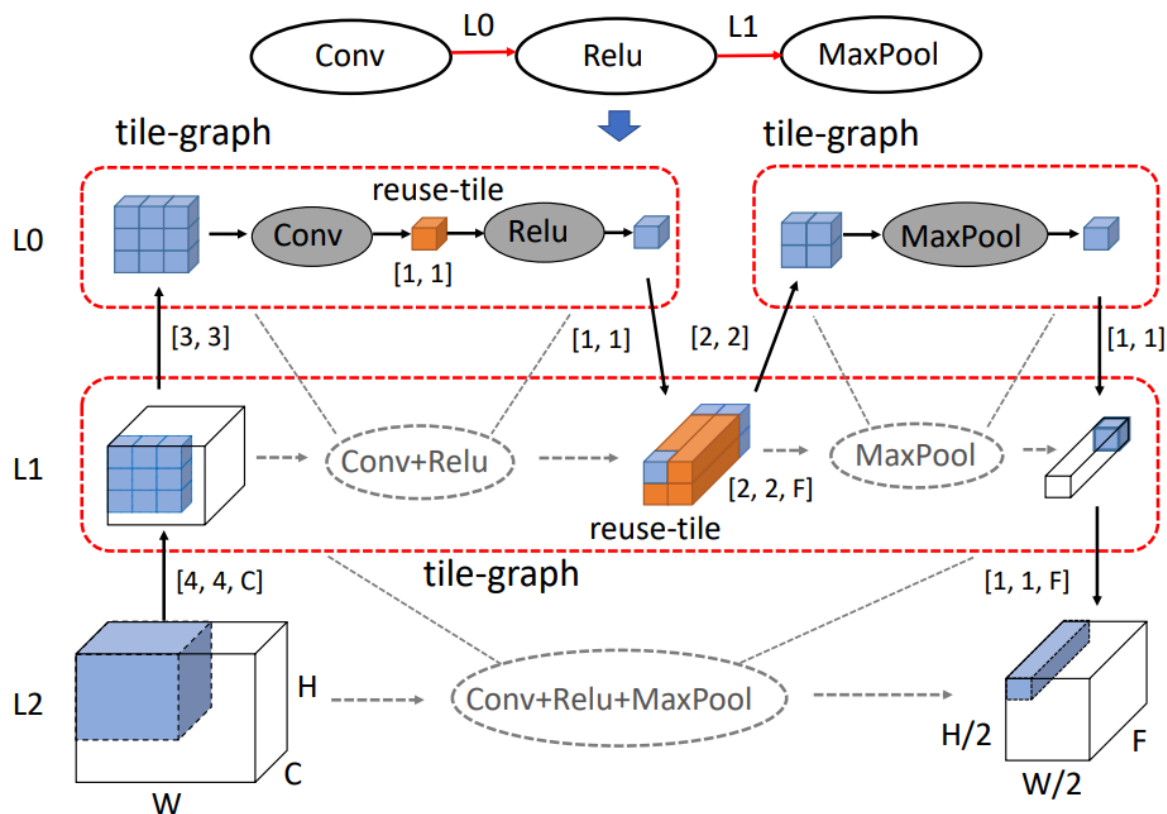
3.2 Fusion

框架	决策依据
TensorRT	算子库是否有对应融合实现, 如果有则匹配
TVM	1. 如果走CUTLASS/cuBLAS, 则FuseOpsByPattern 2. 否则, FuseOps a. 算子分类OpPatternKind, 数值越小, 越容易融合 📖 holo-graph-compiler b. 构建后支配树 c. 根据规则, 判断当前节点是否可以跟它的直接后支配点融合
IREE	1. 将能合并到同一个linalg.generic的算子合并成一个算子 2. 选取root op, 根据一定规则, 向上融合producer, 向下融合consumer



通用融合方案: *WELDER: Scheduling Deep Learning Memory Access via Tile-graph*
<https://www.usenix.org/conference/osdi23/presentation/shi>

- 1. 相邻算子能否共轴切分决定是否可以融合
- 2. 相邻算子之间在哪一级memory上reuse(融合)
- 3. 输出tile向上反推出整个subgraph的切分, 根据tile-size计算出traffic, 选出最佳的tile-size



3.3 Sharding&Tiling

3.3.1 IREE

IREE里的fusion和tiling不是同步, 而是前后两层: 先决定哪些算子进同一个kernel, 再在codegen里按多层tile size把它们tile-and-fuse进同一套循环. 底层都走MLIR的TilingInterface + `scf::tileConsumerAndFuseProducersUsingSCF`

1. fusion

代码块

```
1 fusion (Linalg IR)
2   1. 将能合并到同一个linalg.generic的算子合并成一个linalg算子
3   2. 选取root op, 根据一定规则, 向上融合producer, 向下融合consumer, 扩张fusion group
4
```

2. lowering_config

给每个op附上切分的config info

代码块

```
1 func.func @fused_matmul_bias_relu(
2     %A: tensor<128x256xf32>,
3     %B: tensor<256x64xf32>,
4     %bias: tensor<64xf32>
5 ) -> tensor<128x64xf32> {
6     %cst = arith.constant 0.0 : f32
7
8     // matmul
9     %empty = tensor.empty() : tensor<128x64xf32>
10    %fill = linalg.fill ins(%cst : f32) outs(%empty : tensor<128x64xf32>)
11        -> tensor<128x64xf32>
12
13    %mm = linalg.matmul
14        {lowering_config = #iree_cpu.lowering_config<
15        distribution = [64, 64, 0],
16        vector_common_parallel = [8, 16, 0],
17        vector_reduction = [0, 0, 32]>}
18        ins(%A, %B : tensor<128x256xf32>, tensor<256x64xf32>)
19        outs(%fill : tensor<128x64xf32>) -> tensor<128x64xf32>
20
21    // add
22    %add_init = tensor.empty() : tensor<128x64xf32>
23    %add = linalg.generic {
24        indexing_maps = [affine_map<(d0, d1) -> (d0, d1)>,
25                        affine_map<(d0, d1) -> (d1)>,
26                        affine_map<(d0, d1) -> (d0, d1)>],
27        iterator_types = ["parallel", "parallel"]}
28        ins(%mm, %bias : tensor<128x64xf32>, tensor<64xf32>)
29        outs(%add_init : tensor<128x64xf32>)
30        attrs = {lowering_config = #iree_cpu.lowering_config<
31        distribution = [64, 64],
32        vector_common_parallel = [8, 16]>} {
33        ^bb0(%x: f32, %b: f32, %o: f32):
34            %t = arith.addf %x, %b : f32
35            linalg.yield %t : f32
36    } -> tensor<128x64xf32>
37
38    // relu
39    %relu_init = tensor.empty() : tensor<128x64xf32>
40    %relu = linalg.generic {
41        indexing_maps = [affine_map<(d0, d1) -> (d0, d1)>,
42                        affine_map<(d0, d1) -> (d0, d1)>],
43        iterator_types = ["parallel", "parallel"]}
44        ins(%add : tensor<128x64xf32>) outs(%relu_init : tensor<128x64xf32>)
45        attrs = {lowering_config = #iree_cpu.lowering_config<
```

```

46     distribution = [64, 64],
47     vector_common_parallel = [8, 16]>} {
48     ^bb0(%x: f32, %o: f32):
49         %t = arith.maximumpf %x, %cst : f32
50         linalg.yield %t : f32
51     } -> tensor<128x64xf32>
52
53     return %relu : tensor<128x64xf32>
54 }

```

3. 找到root op, 根据lowering_config的distribution进行sharding

代码块

```

1  %result = scf.forall (%m, %n) in (128, 64) step (64, 64)
2      shared_outs(%out = %relu_init) -> (tensor<128x64xf32>) {
3      %add_tile = tensor.extract_slice %add[%m, %n] [64, 64] [1, 1]
4      %out_tile = tensor.extract_slice %out[%m, %n] [64, 64] [1, 1]
5      %relu_tile = linalg.generic ...
6          ins(%add_tile : tensor<64x64xf32>)
7          outs(%out_tile : tensor<64x64xf32>) -> tensor<64x64xf32>
8      scf.forall.in_parallel {
9          tensor.parallel_insert_slice %relu_tile into %out[%m, %n] [64, 64] [1, 1]
10     }
11 } {mapping = [#iree_codegen.workgroup_mapping<y>,
12             #iree_codegen.workgroup_mapping<x>]}

```

4. 向上fuse producer, 向下fuse consumer

relu sharding后, 它input的extract_slice的输入是add算子, 所以根据该extract_slice的拆分配置, 继续对add进行拆分, 依此类推

代码块

```

1  %result = scf.forall (%m, %n) in (128, 64) step (64, 64)
2      shared_outs(%out = %empty) -> (tensor<128x64xf32>) {
3      %A_t = tensor.extract_slice %A[%m, 0] [64, 256] [1, 1] // 一整行 K
4      %B_t = tensor.extract_slice %B[0, %n] [256, 64] [1, 1]
5      %out_t = tensor.extract_slice %out[%m, %n] [64, 64] [1, 1]
6      %fill_t = linalg.fill ins(%cst : f32) outs(%out_t)
7      %mm_t = linalg.matmul ins(%A_t, %B_t) outs(%fill_t) // 64x64x256
8      %bias_t = tensor.extract_slice %bias[%n] [64] [1]
9      %add_t = linalg.generic ins(%mm_t, %bias_t) ... // 64x64

```

```

10    %relu_t = linalg.generic ins(%add_t) ...
11    scf.forall.in_parallel {
12        tensor.parallel_insert_slice %relu_t into %out[%m, %n] [64, 64] [1, 1]
13    }
14 } {mapping = [#iree_codegen.workgroup_mapping<y>,
15             #iree_codegen.workgroup_mapping<x>]}

```

5. 根据lowering_config的vector_common_parallel, 进行二次tiling

代码块

```

1  scf.forall (%m, %n) in (128, 64) step (64, 64) {      // workgroup
2      %A_t = extract_slice %A[%m, 0] [64, 256]
3      %B_t = extract_slice %B[0, %n] [256, 64]
4      %out_t = extract_slice %out[%m, %n] [64, 64]
5
6      %wg_out = scf.forall (%i, %j) in (64, 64) step (8, 16)
7          shared_outs(%acc = %out_t) -> tensor<64x64xf32> {
8              %A_tt = extract_slice %A_t[%i, 0] [8, 256]
9              %B_tt = extract_slice %B_t[0, %j] [256, 16]
10             %acc_t = extract_slice %acc[%i, %j] [8, 16]
11             %fill_t = linalg.fill ins(%cst) outs(%acc_t)
12             %mm_tt = linalg.matmul ins(%A_tt, %B_tt) outs(%fill_t) // 8x16x256
13             %bias_tt = extract_slice %bias[%n + %j] [16]
14             %add_tt = linalg.generic ins(%mm_tt, %bias_tt) ... // 8x16
15             %relu_tt = linalg.generic ins(%add_tt) ...
16             scf.forall.in_parallel {
17                 parallel_insert_slice %relu_tt into %acc[%i, %j] [8, 16]
18             }
19         }
20     parallel_insert_slice %wg_out into %out[%m, %n] [64, 64]
21 }

```

6. 根据lowering_config的vector_reduction, 对matmul的k轴进行切分

代码块

```

1  scf.forall (%i, %j) in (64, 64) step (8, 16) shared_outs(%acc = %out_t) {
2      %acc_t = extract_slice %acc[%i, %j] [8, 16]
3      %fill_t = linalg.fill ins(%cst) outs(%acc_t) // K 循环外
4
5      %mm_tt = scf.for %k = 0 to 256 step 32
6          iter_args(%acc_k = %fill_t) -> tensor<8x16xf32> {

```

```

7      %A_k = extract_slice %A_t[%i, %k] [8, 32]
8      %B_k = extract_slice %B_t[%k, %j] [32, 16]
9      %mm_k = linalg.matmul ins(%A_k, %B_k) outs(%acc_k) // 8x16x32
10     scf.yield %mm_k
11 }
12
13 %bias_tt = extract_slice %bias[%n + %j] [16]
14 %add_tt = linalg.generic ins(%mm_tt, %bias_tt) ...
15 %relu_tt = linalg.generic ins(%add_tt) ...
16 scf.forall.in_parallel {
17     parallel_insert_slice %relu_tt into %acc[%i, %j] [8, 16]
18 }
19 }

```

● lowering_config是如何生成的

代码块

```

1  MaterializeUserConfigs // 用户/tuning已经写好的config
2      tuning spec中检索(root op type + shape)
3      ↓ 若没有
4      SelectLoweringStrategy(启发式)
5      选root op
6      按op 类型 + target算出tile
7      挂lowering_config到op上
8      把config传播到同dispatch其它compute op
9      ↓
10  开始 TileAndDistribute/TileAndFuse

```

Tunner工作流程

代码块

```

1  knobs: 组合空间
2  SMT: 自动求解器
3
4  // 离线tuning
5  1. 构造knobs
6  2. SMT根据约束, 搜索合法解
7  3. 编译, 上机测试, 记录该合法解性能参数

```

3.3.2 TVM

tvm自动调度架构的演进

代码块

- 1 第一代：AutoTVM(基于模板的调优)
- 2 ↓ 问题：需要人工为每个算子编写schedule template
- 3 第二代：AutoScheduler/Ansor（无模板自动调优）
- 4 ↓ 问题：与AutoTVM的API体系隔离，难以扩展新的调度原语
- 5 第三代：MetaSchedule(统一框架，同时支持人工模板和自动调优) + DLight(轻量规则)

DLight

基于PrimFunc进行调度, 纯计算语义描述

代码块

```

1  @T.prim_func(s_tir=True)
2  def fused_matmul_add_relu(x, weight, bias, out):
3      # 未调度: plain loop, 无 thread binding, 无 cache buffer
4      for i, j, k in T.grid(1, 768, 768):
5          with T.sblock("matmul"):
6              vi, vj, vk = T.axis.remap("SSR", [i, j, k])
7              with T.init():
8                  out[vi, vj] = T.float32(0)
9                  out[vi, vj] += x[vi, vk] * weight[vk, vj]
10
11     for i, j in T.grid(1, 768):
12         with T.sblock("relu"):
13             vi, vj = T.axis.remap("SS", [i, j])
14             out[vi, vj] = T.max(out[vi, vj] + bias[vj], 0)

```

1. 按照如下顺序匹配调度模式, 若匹配成功, 则按照该模式的split/bind/cache规则调度这个主block; 如果调度失败, 则继续往下匹配, 若所有匹配的调度模式都失败, 则使用Fallback()兜底.

```

1 // 用户可自定义顺序
2 // 下面调度规则的匹配是根据 "block轴类型 + 读写索引/pattern"判断，不是算子名
3 // 每种调度规则是写死的
4 dl.ApplyDefaultSchedule(
5     dl.gpu.Matmul(),
6     dl.gpu.GEMV(),
7     dl.gpu.LowBatchGEMV(),
8     dl.gpu.Transpose(),
9     dl.gpu.RMSNorm(),
10    dl.gpu.Reduction(),
11    dl.gpu.GeneralReduction(),
12    dl.gpu.Fallback(),
13 )

```

规则

代码块

```

1 ApplyDefaultSchedule
2   └─ rule.apply() 成功
3       |
4       └─ Matmul    reindex → layout → (可选 tensorize)
5                   pad → split/reorder/bind
6                   cache_write local → reverse_compute_at
7                   cache_read shared + compute_at(ko)
8                   inline producers/consumers
9                   decompose_reduction(ko)
10          |
11          └─ GEMV    fuse/split → rfactor ×2 → bind
12                  cache_read local/shared
13                  reverse_compute_at 两段规约
14                  decompose_reduction
15                  reverse_compute_at(epilogue)
16          |
17          └─ Reduction split 规约轴 → rfactor → bind
18                  decompose → reverse_compute_at 写回/epilogue
19          |
20          └─ GeneralReduction
21                  fuse S→bx, split R→tx
22                  中间结果 set_scope shared + compute_at(bx)
23          |
24          └─ Fallback fuse S → bind bx/tx, 有 R 则 decompose

```

2. 生成IR

代码块

```
1  @T.prim_func(s_tir=True)
2  def fused_matmul_add_relu(x, weight, bias, out):
3      T.func_attr({"tirx.is_scheduled": True})
4
5      # 16 路 K 部分和 (每个 thread 一份)
6      rf = T.sblock_alloc_buffer((16, 1, 768), scope="local")
7      # 规约完成后的完整结果
8      acc = T.sblock_alloc_buffer((1, 768), scope="local")
9
10     # 48 个 block × 每 block 16 个输出列 = 768
11     for tile in T.thread_binding(48, thread="blockIdx.x"):
12         for tx in T.thread_binding(16, thread="threadIdx.x"):      # 输出列
13             for ty in T.thread_binding(16, thread="threadIdx.y"):  # K分段
14                 j = tile * 16 + tx
15
16                 # 本thread 的部分K累加: k = ty, 16+ty, ..., 752+ty
17                 with T.sblock("matmul_rf_init"):
18                     rf[ty, 0, j] = T.float32(0)
19                 for k_tile in range(48):
20                     with T.sblock("matmul_rf_update"):
21                         k = k_tile * 16 + ty
22                         rf[ty, 0, j] += x[0, k] * weight[k, j]
23
24                 # 跨thread把16路部分和加全
25                 for tx in T.thread_binding(16, thread="threadIdx.x"):
26                     for ty in T.thread_binding(16, thread="threadIdx.y"):
27                         j = tile * 16 + tx
28                         with T.sblock("matmul"):
29                             vj = T.axis.spatial(768, j)
30                             vty = T.axis.reduce(16, ty)
31                             with T.init():
32                                 acc[0, vj] = T.float32(0)
33                                 acc[0, vj] += rf[vty, 0, vj]
34
35                 # 完整累加结果上再 bias + relu
36                 for tx in T.thread_binding(16, thread="threadIdx.x"):
37                     j = tile * 16 + tx
38                     with T.sblock("relu"):
39                         out[0, j] = T.max(acc[0, j] + bias[j], 0)
```

1. 生成搜索空间

sketch

代码块

```
1  # matmul:
2  SamplePerfectTile(i, n=4)          # 搜索点
3  SamplePerfectTile(j, n=4)
4  SamplePerfectTile(k, n=2)
5  split + reorder(SSSRRSRS)
6  bind blockIdx.x / vthread.x / threadIdx.x
7  cache_write local; cache_read shared
```

给搜索点生成一组采样解

代码块

```
1  ("SamplePerfectTile", [4, 8, 8, 24])
2  ("SamplePerfectTile", [4, 8, 8, 24])
3  ("SamplePerfectTile", [48, 16])
```

2. 上机测试迭代

代码块

```
1  // task: PrimFunc
2
3  while 已测次数 < max_trials_global:
4      1. 选取task(优先选择整图中调用次数多, 迭代收益大, latency较大的task)
5      2. candidates = task.strategy.GenerateMeasureCandidates() # 采用遗传算法, 生成迭代候选方案
6      3. 若候选方案为空或者达到最大迭代次数, 则退出
7      4. Builder 编译这批 IR
8      5. Runner 在 GPU 上测 latency
9      6. strategy.NotifyRunnerResults(...)
10     7. Database 写入 (workload, trace, run_secs)
11     8. CostModel.Update(...) # 用实测更新 XGBoost
```

3. 实际推理

workload哈希是对一份IRModule(未调度的PrimFunc)做结构哈希, 用来在数据库里当主键.

按workload哈希找最好的record, 使用该record的调度策略, 得到已调度PrimFunc. 若找不到对应的record, 则会再跑一遍DLight兜底.

4. 附录

tvm融合算子分类

类型	编号	含义	算子举例
kElemWise	0	输出一点 ---> 输入同一点	add, relu
kBroadcast	1	轴顺序不变, 可广播	bias_add
kInjective	2	输出一点只依赖一个输入点, 映射可复杂	reshape, concat, transpose
kCommReduce	3	多对一归约	sum, max, mean
kOutEWiseFusabl e	4	大算子, 只允许后面接elementwise	conv2d, matmul
kTuple	7	将多个SSA打包成tuple, 送给concat	无具体算子
kOpaque	8	其它	外部调用, scatter, sort